

Dokumentacja Django

Spis treści

- 1. Wprowadzenie do Django
 - 1.1. Czym jest Django — framework webowy, architektura MTV
 - 1.2. Architektura MTV (Model-Template-View) — czym się różni od MVC
 - 1.3. Dlaczego Django — zalety frameworka
 - 1.4. Wymagania — Python 3.10+, pip, venv
- 2. Instalacja i pierwszy projekt
 - 2.1. Tworzenie środowiska wirtualnego (venv) — krok po kroku
 - 2.2. Instalacja Django przez pip
 - 2.3. Tworzenie projektu — django-admin startproject
 - 2.4. Struktura plików projektu (manage.py, settings.py, urls.py, wsgi.py, asgi.py)
 - 2.5. Uruchomienie serwera deweloperskiego — python manage.py runserver
 - 2.6. Tworzenie aplikacji — python manage.py startapp
 - 2.7. Rejestracja aplikacji w INSTALLED_APPS
 - 2.8. Struktura plików aplikacji (models.py, views.py, urls.py, admin.py, apps.py, tests.py)
- 3. Widoki (Views) — serce logiki aplikacji
 - 3.1. Czym jest widok — funkcja przyjmująca request, zwracająca response
 - 3.2. Pierwszy widok — HttpResponse("Hello World")
 - 3.3. Routing URL — path(), include(), patterns
 - 3.4. Parametry w URL — konwertery int:pk, str:slug, uuid:id
 - 3.5. Nazwy URL — name='nazwa' i reverse()
 - 3.6. Przekierowania — redirect(), reverse()
 - 3.7. Kody odpowiedzi HTTP — 200, 301, 302, 404, 500
 - 3.8. Widoki oparte na klasach (CBV) vs widoki funkcyjne (FBV) — porównanie
- 4. Szablony (Templates) — warstwa prezentacji
 - 4.1. Konfiguracja szablonów
 - 4.2. Renderowanie szablonu
 - 4.3. Język szablonów Django (DTL) — zmienne
 - 4.4. Tagi szablonów
 - 4.5. Filtry szablonów
 - 4.6. Dziedziczenie szablonów
 - 4.7. Dołączanie fragmentów
 - 4.8. Pliki statyczne (CSS, JS, obrazy)
 - 4.9. Komentarze w szablonach
- 5. Modele (Models) — warstwa danych
 - 5.1. Czym jest model
 - 5.2. Definiowanie modelu
 - 5.3. Typy pól
 - 5.4. Opcje pól
 - 5.5. Klucz główny (Primary Key)
 - 5.6. Migracje
 - 5.7. Relacje: Wiele do jednego (ForeignKey)
 - 5.8. Relacje: Wiele do wielu (ManyToManyField)
 - 5.9. Relacje: Jeden do jednego (OneToOneField)
 - 5.10. Opcja on_delete
 - 5.11. Praca z istniejącą bazą danych (inspectdb)
- 6. ORM — zapytania do bazy danych

- 6.1. Czym jest ORM
- 6.2. Tworzenie obiektów
- 6.3. Pobieranie wszystkich rekordów
- 6.4. Pobieranie jednego rekordu
- 6.5. Filtrowanie
- 6.6. Lookup'y — zaawansowane filtrowanie
- 6.7. Sortowanie
- 6.8. Ograniczanie wyników
- 6.9. Aktualizacja rekordów
- 6.10. Usuwanie rekordów
- 6.11. Łączenie zapytań (chaining)
- 6.12. Zapytania na relacjach
- 7. Panel administracyjny (Django Admin)
 - 7.1. Tworzenie superużytkownika
 - 7.2. Rejestrowanie modeli w admin.py
 - 7.3. Klasa ModelAdmin
 - 7.4. Edycja widoku szczegółów
- 8. Formularze (Forms)
 - 8.1. Czym jest formularz
 - 8.2. Definiowanie formularza
 - 8.3. Obsługa formularza w widoku (FBV)
 - 8.4. Wyświetlanie formularza w szablonie
 - 8.5. Ręczne renderowanie formularza
 - 8.6. ModelForm — szybki formularz na bazie modelu
 - 8.7. Formularz z przesyłaniem plików
- 9. Widoki oparte na klasach (Class-Based Views)
 - 9.1. Dlaczego CBV? FBV vs CBV
 - 9.2. TemplateView — wyświetlanie szablonu
 - 9.3. ListView — lista obiektów
 - 9.4. DetailView — szczegóły jednego obiektu
 - 9.5. CreateView — tworzenie nowego obiektu
 - 9.6. UpdateView — edycja istniejącego obiektu
 - 9.7. DeleteView — usuwanie obiektu
 - 9.8. FormView — obsługa zwykłego formularza
 - 9.9. Podsumowanie — magii CBV
- 10. Uwierzytelnianie i autoryzacja
 - 10.1. Wbudowany system auth
 - 10.2. Wbudowane formularze logowania i wylogowanie
 - 10.3. Rejestracja użytkownika — UserCreationForm
 - 10.4. Ograniczanie dostępu do stron dla niezalogowanych
 - 10.5. Grupy i uprawnienia dla pracowników
 - 10.6. Dostęp do użytkownika w szablonie
 - 10.7. Rozszerzanie modelu User — OneToOne
- 11. Pliki statyczne i media
 - 11.1. Różnica między static a media
 - 11.2. Konfiguracja plików statycznych
 - 11.3. Używanie CSS i JS w szablonie
 - 11.4. Konfiguracja plików Media (Wgrywanych)
 - 11.5. Serwowanie plików Media na serwerze deweloperskim
- 12. Środowisko uruchomieniowe i bezpieczeństwo

- 12.1. Zmienna DEBUG i ukrywanie błędów
 - 12.2. Secret Key (Klucz kryptograficzny)
 - 12.3. Ochrona CSRF (Cross Site Request Forgery)
 - 13. Sesje (Sessions) i Wiadomości Flash (Messages)
 - 13.1. Przechowywanie danych tymczasowych (Sesja)
 - 13.2. Wbudowany moduł wiadomości (Messages framework)
 - 14. Sygnały w Django (Signals)
 - 14.1. Co rozwiązują Sygnały
 - 14.2. Jak "Łapać sygnał" - tworzenie powiązań
 - 15. Testowanie automatyczne (Testing)
 - 15.1. Idea testów
 - 15.2. Pierwszy prosty test
 - 15.3. Testowanie Widoków - Client
 - 16. Zaawansowane zapytania i Optymalizacja bazy danych
 - 16.1. Problem N+1 zapytań
 - 16.2. select_related (dla relacji "Jeden")
 - 16.3. prefetch_related (dla relacji "Wiele")
 - 17. Formsety — wielokrotne formularze
 - 17.1. Czym jest Formset?
 - 17.2. ModelFormsets - Magia dla modeli hurtowych
 - 18. Paginacja — stronicowanie
 - 19. REST API i Django REST Framework (DRF) — w skrócie
 - 20. Architektura serwerowa i publikowanie systemu na świat
 - 21. Praktyczne przykłady – tworzenie aplikacji od A do Z
 - Przykład 1: Prosta Lista Zadań (To-Do List)
 - Przykład 2: Blog z relacjami (Kategorie i Artykuły)
 - Przykład 3: Miniforum — Rejestracja i Wątki Użytkowników
 - Przykład 4: Prosty Dziennik Szkolny (Od zera do finału)
-

1. Wprowadzenie do Django

Django to jeden z najpopularniejszych frameworków webowych napisanych w języku Python. Został stworzony w 2003 roku przez programistów gazety Lawrence Journal-World i udostępniony publicznie w 2005 roku na licencji BSD. Nazwa frameworka pochodzi od imienia legendarnego gitarzysty jazzowego — Django Reinhardta. Od tego czasu Django stało się jednym z najczęściej wybieranych narzędzi do tworzenia aplikacji internetowych, zarówno prostych stron, jak i rozbudowanych systemów (Instagram, Pinterest, Mozilla, Disqus czy Bitbucket korzystają z Django).

1.1. Czym jest Django — framework webowy, architektura MTV

Django jest **wysokopoziomowym frameworkiem webowym** napisanym w Pythonie, który umożliwia szybkie tworzenie bezpiecznych i łatwych w utrzymaniu aplikacji internetowych. Główną filozofią Django jest zasada **DRY** (Don't Repeat Yourself) — czyli unikanie powtarzania kodu — oraz „**batteries included**” (baterie w zestawie), co oznacza, że framework dostarcza gotowe narzędzia do większości typowych zadań w tworzeniu aplikacji webowych.

Framework webowy to zbiór bibliotek, narzędzi i konwencji, które ułatwiają budowanie aplikacji internetowych. Zamiast pisać wszystko od zera (obsługę HTTP, routing, dostęp do bazy danych, system szablonów, zarządzanie sesjami, uwierzytelnianie), programista korzysta z gotowych, przetestowanych komponentów.

Django realizuje architekturę **MTV (Model-Template-View)**, która jest wariacją klasycznego wzorca **MVC (Model-View-Controller)**. W Django:

- **Model** — definiuje strukturę danych (tabele w bazie danych). Odpowiada za logikę danych, walidację i relacje między tabelami. Każdy model to klasa Pythona, która mapuje się na tabelę w bazie.

- **Template (Szablon)** — odpowiada za warstwę prezentacji, czyli to, co widzi użytkownik w przeglądarce. Szablony to pliki HTML z osadzonym językiem szablonów Django (DTL — Django Template Language).
- **View (Widok)** — pełni rolę pośrednika pomiędzy modelem a szablonem. Widok odbiera żądanie HTTP od użytkownika, przetwarza dane (pobiera je z bazy, wykonuje obliczenia), a następnie zwraca odpowiedź HTTP (najczęściej wyrenderowany szablon HTML).

Schemat działania Django (cykl request-response):



Kluczowe cechy Django:

CECHA	OPIS
ORM (Object-Relational Mapping)	Dostęp do bazy danych przez obiekty Pythona, bez pisania SQL
Panel administracyjny	Automatycznie generowany panel do zarządzania danymi
System szablonów	Własny język szablonów (DTL) do generowania HTML
Routing URL	Elastyczny system mapowania adresów URL na widoki
Bezpieczeństwo	Wbudowana ochrona przed CSRF, XSS, SQL Injection
System migracji	Automatyczne zarządzanie zmianami w bazie danych
System uwierzytelniania	Gotowy system logowania, rejestracji, uprawnień
Internacjonalizacja (i18n)	Wbudowane wsparcie dla tłumaczeń i wielu języków
Cache	Wbudowany system cachowania (pamięć podręczna)
Middleware	Przetwarzanie żądań/odpowiedzi na wielu poziomach

Przykład minimalnej aplikacji Django — widok zwracający tekst:

```
# views.py – najprostszy widok Django
from django.http import HttpResponse

def strona_glowna(request):
    """
    Widok przyjmuje obiekt request (żądanie HTTP)
    i zwraca obiekt HttpResponse (odpowiedź HTTP).
    To jest najprostszy możliwy widok w Django.
    """
    return HttpResponse("Witaj w Django!")
```

```
# urls.py – mapowanie URL na widok
from django.urls import path
from . import views

urlpatterns = [
    path('', views.strona_glowna, name='strona_glowna'),
]
```

Django jest frameworkiem **synchronicznym** (domyślnie), ale od wersji 3.1 wspiera również **widoki asynchroniczne** (async views). W wersji 5.x wsparcie dla asynchroniczności jest znacznie rozbudowane.

1.2. Architektura MTV (Model-Template-View) — czym się różni od MVC

Architektura **MTV** stosowana w Django jest bezpośrednio inspirowana wzorcem **MVC (Model-View-Controller)**, który jest jednym z najstarszych i najpopularniejszych wzorców architektonicznych w programowaniu. Aby dobrze zrozumieć Django, warto poznać różnice i podobieństwa między tymi dwoma podejściami.

Wzorzec MVC (Model-View-Controller):

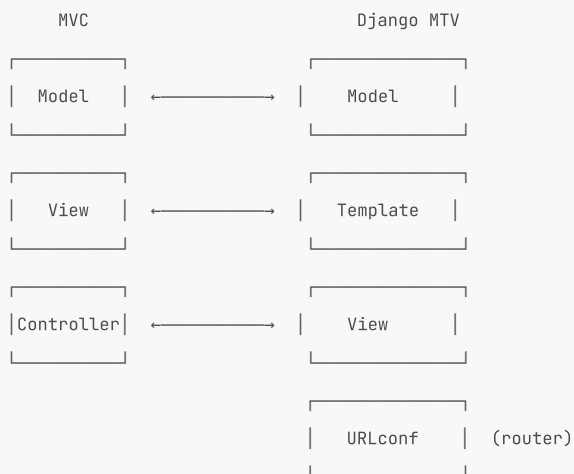
WARSTWA	ODPOWIEDZIALNOŚĆ
Model	Logika danych — definiuje strukturę danych, reguły biznesowe, walidację
View (Widok)	Warstwa prezentacji — to, co widzi użytkownik (HTML, CSS)
Controller (Kontroler)	Pośrednik — odbiera żądanie od użytkownika, przetwarza je, wywołuje model i widok

Wzorzec MTV w Django (Model-Template-View):

WARSTWA MTV	ODPOWIEDNIK W MVC	ODPOWIEDZIALNOŚĆ W DJANGO
Model	Model	Definicja struktury danych, relacji, walidacji. Klasy w <code>models.py</code>
Template	View (Widok)	Warstwa prezentacji — pliki HTML z językiem szablonów DTL
View	Controller	Logika aplikacji — funkcje/klasy w <code>views.py</code> , przetwarzanie żądań

Kluczowa różnica polega na nazewnictwie:

- W MVC **View** = warstwa prezentacji (HTML). W Django odpowiednikiem jest **Template**.
- W MVC **Controller** = logika przetwarzania żądań. W Django odpowiednikiem jest **View**.
- Dodatkowo w Django istnieje **URLconf** (plik `urls.py`), który pełni rolę **routera** — nie ma bezpośredniego odpowiednika w klasycznym MVC.



Pełny przepływ danych w Django MTV — krok po kroku:

1. **Użytkownik** wpisuje adres URL w przeglądarce, np. `http://localhost:8000/artykuly/5/`.
2. **URLconf** (`urls.py`) analizuje URL i dopasowuje go do odpowiedniego widoku.
3. **View** (`views.py`) — widok odbiera obiekt `request`, pobiera dane z modelu, przetwarza logikę biznesową.
4. **Model** (`models.py`) — widok komunikuje się z bazą danych przez ORM, np. pobiera artykuł o `id=5`.
5. **Template** (`artykul.html`) — widok przekazuje dane do szablonu, który renderuje HTML.
6. **Odpowiedź HTTP** — wyrenderowany HTML jest zwracany do przeglądarki użytkownika.

```
# Kompletny przykład przepływu MTV

# == models.py ==
from django.db import models

class Artykuł(models.Model):
    """Model reprezentujący artykuł w bazie danych (warstwa Model)."""
    tytuł = models.CharField(max_length=200, verbose_name="Tytuł")
    tresc = models.TextField(verbose_name="Treść")
    data_publicacji = models.DateTimeField(auto_now_add=True, verbose_name="Data publikacji")

    class Meta:
        verbose_name = "Artykuł"
        verbose_name_plural = "Artykuły"
        ordering = ['-data_publicacji']

    def __str__(self):
        return self.tytuł

# == views.py ==
from django.shortcuts import render, get_object_or_404
from .models import Artykuł

def szczegoly_artykulu(request, pk):
    """
    Widok (warstwa View/Controller) – pobiera artykuł z bazy
    i przekazuje go do szablonu.
    """
```

```

"""

artykul = get_object_or_404(Artykul, pk=pk)
context = {
    'artykul': artykul,
}

return render(request, 'blog/artykul_szczegoly.html', context)

# == urls.py ==
from django.urls import path
from . import views

urlpatterns = [
    path('artykuly/<int:pk>/', views.szczegoly_artykulu, name='szczegoly_artykulu'),
]

```

```

<!-- == templates/blog/artykul_szczegoly.html == -->
<!-- Szablon (warstwa Template/View) – prezentacja danych -->
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <title>{{ artykul.tytul }}</title>
</head>
<body>
    <article>
        <h1>{{ artykul.tytul }}</h1>
        <p class="data">Opublikowano: {{ artykul.data_publicacji|date:"d.m.Y H:i" }}</p>
        <div class="tresc">
            {{ artykul.tresc }}
        </div>
    </article>
    <a href="{% url 'lista_artykulow' %}">- Powrót do listy</a>
</body>
</html>

```

Warto pamiętać, że w Django **sam framework pełni rolę kontrolera** — zajmuje się routowaniem (URLconf), przetwarzaniem middleware, obsługą sesji i innymi niskopoziomowymi aspektami obsługi żądań HTTP. Programista skupia się na pisaniu modeli, widoków i szablonów.

1.3. Dlaczego Django — zalety frameworka

Django wyróżnia się spośród innych frameworków webowych wieloma zaletami, które sprawiają, że jest doskonałym wyborem zarówno dla początkujących, jak i dla doświadczonych programistów.

Główne zalety Django:

ZALETY	OPIS
ORM	Obiektowy dostęp do bazy danych — nie trzeba pisać SQL ręcznie
Panel administracyjny	Automatycznie generowany interfejs do zarządzania danymi
Bezpieczeństwo	Wbudowana ochrona przed najczęstszymi atakami webowymi
System migracji	Automatyczne śledzenie i aplikowanie zmian w strukturze bazy
Skalowalność	Sprawdzony w ogromnych serwisach (Instagram, Pinterest)
Dokumentacja	Jedna z najlepszych dokumentacji wśród frameworków open source
Społeczność	Ogromna społeczność, tysiące pakietów zewnętrznych (Django Packages)
DRY	Filozofia „Don't Repeat Yourself” — minimalizacja powtórzeń kodu
Batteries included	Wszystko, czego potrzebujesz, jest w zestawie
Szybki rozwój	Szybkie prototypowanie i rozwijanie aplikacji

1. ORM (Object-Relational Mapping):

ORM w Django pozwala na pracę z bazą danych za pomocą obiektów Pythona, bez konieczności pisania surowego SQL. Każdy model (klasa Pythona) odpowiada tabeli w bazie danych, a każde pole modelu odpowiada kolumnie.

```
# Zamiast pisać SQL:
# SELECT * FROM artykuł WHERE opublikowany = true ORDER BY data_publicacji DESC;

# W Django piszemy:
from blog.models import Artykuł

# Pobranie wszystkich opublikowanych artykułów, posortowanych od najnowszych
artykuły = Artykuł.objects.filter(opublikowany=True).order_by('-data_publicacji')

# Tworzenie nowego rekordu
nowy_artykuł = Artykuł.objects.create(
    tytuł="Mój pierwszy artykuł",
    treść="Treść artykułu...",
    opublikowany=True
)

# Aktualizacja rekordu
artykuł = Artykuł.objects.get(pk=1)
artykuł.tytuł = "Zmieniony tytuł"
artykuł.save()

# Usunięcie rekordu
artykuł.delete()
```

2. Panel administracyjny (Django Admin):

Django automatycznie generuje panel administracyjny, który pozwala na zarządzanie danymi w bazie bez pisania dodatkowego kodu. Wystarczy zarejestrować model w pliku `admin.py`.

```
# admin.py – rejestracja modelu w panelu administracyjnym
from django.contrib import admin
from .models import Artykuł
```



```
@admin.register(Artykuł)

class ArtykułAdmin(admin.ModelAdmin):
    """Konfiguracja panelu admina dla modelu Artykuł."""
    list_display = ['tytuł', 'data_publicacji', 'opublikowany']
    list_filter = ['opublikowany', 'data_publicacji']
    search_fields = ['tytuł', 'tresc']
    date_hierarchy = 'data_publicacji'
    ordering = ['-data_publicacji']
```

3. Bezpieczeństwo:

Django chroni przed najczęstszymi atakami webowymi:

ATAK	OCHRONA W DJANGO
CSRF (Cross-Site Request Forgery)	Token CSRF dodawany automatycznie do formularzy (<code>{% csrf_token %}</code>)
XSS (Cross-Site Scripting)	Automatyczne escapowanie zmiennych w szablonach
SQL Injection	ORM parametryzuje zapytania automatycznie
Clickjacking	Middleware <code>X-Frame-Options</code> blokuje osadzanie w iframe
Zarządzanie hasłami	Hasła hashowane algorytmem PBKDF2 z solą
HTTPS	Ustawienia wymuszające HTTPS w produkcji

4. System migracji:

Migracje to mechanizm automatycznego śledzenia zmian w modelach i stosowania ich w bazie danych. Nie trzeba ręcznie modyfikować tabel SQL — Django generuje i wykonuje migracje automatycznie.

```
# Tworzenie migracji na podstawie zmian w modelach
python manage.py makemigrations

# Zastosowanie migracji do bazy danych
python manage.py migrate

# Podgląd stanu migracji
python manage.py showmigrations

# Podgląd wygenerowanego SQL
python manage.py sqlmigrate blog 0001
```

5. Porównanie Django z innymi frameworkami:

CECHA	DJANGO (PYTHON)	FLASK (PYTHON)	EXPRESS.JS (NODE.JS)	LARAVEL (PHP)
Typ	Full-stack	Micro	Minimal	Full-stack
ORM	Wbudowany	Brak (SQLAlchemy)	Brak (Sequelize)	Wbudowany (Eloquent)
Admin panel	Wbudowany	Brak	Brak	Brak (pakiet zewn.)
Szablony	DTL (wbudowany)	Jinja2	Brak (EJS, Pug)	Blade
Migracje	Wbudowane	Flask-Migrate	Brak	Wbudowane
Bezpieczeństwo	Wbudowane	Minimalne	Minimalne	Wbudowane
Krzywa uczenia	Średnia	Niska	Niska	Średnia
Spółeczność	Ogromna	Duża	Ogromna	Duża

1.4. Wymagania — Python 3.10+, pip, venv

Aby rozpocząć pracę z Django 5.x, potrzebne są następujące narzędzia:

NARZĘDZIE	MINIMALNA WERSJA	OPIS
Python	3.10+	Interpreter języka Python
pip	21.0+	Menedżer pakietów Pythona (instalowany z Pythonem)
venv	(wbudowane)	Moduł do tworzenia wirtualnych środowisk (wbudowany w Pythona)
Django	5.0+	Framework webowy (instalowany przez pip)

Kompatybilność wersji Django z Pythonem:

WERSJA DJANGO	PYTHON 3.8	PYTHON 3.9	PYTHON 3.10	PYTHON 3.11	PYTHON 3.12	PYTHON 3.13
Django 4.2 LTS	✓	✓	✓	✓	✓	✗
Django 5.0	✗	✗	✓	✓	✓	✗
Django 5.1	✗	✗	✓	✓	✓	✓
Django 5.2 LTS	✗	✗	✓	✓	✓	✓

Sprawdzenie wersji Pythona:

```
# Windows (cmd lub PowerShell)
python --version

# lub
python -V

# Linux / macOS
python3 --version

# lub
python3 -V
```

Wynik powinien wyglądać np.: Python 3.12.4

Sprawdzenie wersji pip:

```
# Windows
pip --version
```

```
# Linux / macOS
pip3 --version
```

Wynik powinien wyglądać np.: `pip 24.0 from /usr/lib/python3.12/site-packages (python 3.12)`

Instalacja Pythona:

Na Windows:

1. Wejdź na stronę python.org/downloads.
2. Pobierz najnowszą wersję Pythona 3.12+.
3. Podczas instalacji **koniecznie zaznacz** opcję „Add Python to PATH” (Dodaj Pythona do zmiennej PATH).
4. Kliknij „Install Now”.

Na Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

Na macOS:

```
# Przez Homebrew
brew install python3
```

Czym jest venv (virtual environment)?

Wirtualne środowisko (venv) to izolowana kopia interpretera Pythona z własnym zestawem zainstalowanych pakietów. Dzięki temu każdy projekt może mieć inne wersje bibliotek, bez konfliktów. Jest to **standardowa praktyka** w pracy z Pythonem i Django — **zawsze** twórz nowe środowisko wirtualne dla każdego projektu.

```
# Tworzenie wirtualnego środowiska
python -m venv moje_srodowisko

# Aktywacja środowiska (Windows)
moje_srodowisko\Scripts\activate

# Aktywacja środowiska (Linux/macOS)
source moje_srodowisko/bin/activate

# Po aktywacji w terminalu pojawi się prefiks:
# (moje_srodowisko) C:\Users\user\projekt>

# Dezaktywacja środowiska
deactivate
```

2. Instalacja i pierwszy projekt

W tym rozdziale krok po kroku utworzymy pierwszy projekt Django — od instalacji frameworka, przez utworzenie projektu i aplikacji, aż po uruchomienie serwera deweloperskiego. Każdy krok jest szczegółowo opisany z poleceniami dla Windows i Linux/macOS.

2.1. Tworzenie środowiska wirtualnego (venv) — krok po kroku

Środowisko wirtualne (venv) to izolowana instalacja Pythona, dedykowana konkretnemu projektowi. Używanie venv jest **obowiązkową praktyką** w profesjonalnym programowaniu w Pythonie. Bez venv pakiety instalowane przez pip trafiają do globalnej instalacji Pythona, co może powodować konflikty wersji między projektami.

Krok 1: Utwórz katalog projektu

```
# Windows
mkdir C:\projekty\moj_projekt
cd C:\projekty\moj_projekt

# Linux / macOS
mkdir -p ~/projekty/moj_projekt
cd ~/projekty/moj_projekt
```

Krok 2: Utwórz wirtualne środowisko

```
# Windows
python -m venv venv

# Linux / macOS
python3 -m venv venv
```

Polecenie `python -m venv venv` tworzy katalog `venv/` w bieżącym folderze, który zawiera:

PLIK / KATALOG	OPIS
<code>venv/bin/</code> (Linux) lub <code>venv/Scripts/</code> (Windows)	Pliki wykonywalne Pythona i pip
<code>venv/lib/</code>	Zainstalowane pakiety Pythona
<code>venv/include/</code>	Pliki nagłówkowe C (do kompilacji rozszerzeń)
<code>venv/pyvenv.cfg</code>	Plik konfiguracyjny środowiska

Krok 3: Aktywuj środowisko wirtualne

```
# Windows (cmd)
venv\Scripts\activate

# Windows (PowerShell)
venv\Scripts\Activate.ps1

# Linux / macOS
source venv/bin/activate
```

Po aktywacji w terminalu pojawi się prefiks z nazwą środowiska:

```
(venv) C:\projekty\moj_projekt>
(venv) user@komputer:~/projekty/moj_projekt$
```

Krok 4: Sprawdź, czy środowisko jest aktywne

```
# Sprawdź, skąd pochodzi Python
# Windows
where python

# Linux / macOS
which python3

# Sprawdź wersję pip w środowisku
pip --version
```

Krok 5: Aktualizacja pip (zalecane)

```
# Aktualizacja pip do najnowszej wersji
pip install --upgrade pip
```

Krok 6: Dezaktywacja środowiska (gdy skończysz pracę)

```
deactivate
```

Ważne uwagi:

- Katalog `venv/` nie powinien być dodawany do repozytorium Git. Dodaj go do `.gitignore`:

```
# .gitignore
venv/
```

- Aby odtworzyć środowisko na innym komputerze, używaj pliku `requirements.txt`:

```
# Zapisanie zainstalowanych pakietów
pip freeze > requirements.txt

# Instalacja pakietów z pliku
pip install -r requirements.txt
```

2.2. Instalacja Django przez pip

Po aktywowaniu środowiska wirtualnego możemy zainstalować Django za pomocą menedżera pakietów **pip**. Pip automatycznie pobierze Django z repozytorium PyPI (Python Package Index) i zainstaluje go razem ze wszystkimi zależnościami.

Instalacja najnowszej stabilnej wersji Django:

```
# Upewnij się, że środowisko wirtualne jest aktywne!
# Powinien być widoczny prefiks (venv)

pip install django
```

Instalacja konkretnej wersji Django:

```
# Instalacja Django 5.2 (konkretna wersja)
pip install django==5.2

# Instalacja Django 5.x (najnowsza z serii 5)
pip install "django ≥ 5.0, < 6.0"
```

Sprawdzenie zainstalowanej wersji Django:

```
# Sposób 1: przez pip
pip show django

# Sposób 2: przez Pythona
python -c "import django; print(django.get_version())"

# Sposób 3: przez django-admin
django-admin --version
```

Przykładowy wynik polecenia `pip show django`:

```
Name: Django
Version: 5.2
Summary: A high-level Python web framework
Home-page: https://www.djangoproject.com/
Author: Django Software Foundation
License: BSD-3-Clause
Location: /home/user/projekty/moj_projekt/venv/lib/python3.12/site-packages
Requires: asgiref, sqlparse
Required-by:
```

Zapisanie zależności do pliku requirements.txt:

```
pip freeze > requirements.txt
```

Plik `requirements.txt` będzie wyglądał mniej więcej tak:

```
asgiref==3.8.1
Django==5.2
sqlparse==0.5.1
```

Sprawdzenie listy zainstalowanych pakietów:

```
pip list
```

PAKIET	WERSJA	OPIS
Django	5.2	Framework webowy
asgiref	3.8.1	Referencja ASGI (serwer asynchroniczny)
sqlparse	0.5.1	Parser SQL (używany wewnętrznie przez Django)

2.3. Tworzenie projektu — django-admin startproject

Projekt Django to zbiór ustawień i konfiguracji dla całej aplikacji webowej. Jeden projekt może zawierać wiele aplikacji (apps). Do tworzenia projektu służy polecenie `django-admin startproject`.

Tworzenie projektu:

```
# Składnia: django-admin startproject <nazwa_projektu> [katalog]
django-admin startproject moj_projekt .
```

Uwaga: Kropka `.` na końcu polecenia oznacza, że projekt zostanie utworzony w **bieżącym katalogu**, bez tworzenia dodatkowego zagnieżdżonego folderu. Bez kropki Django utworzy dodatkowy katalog:

```
# BEZ kropki – tworzy zagnieżdżony katalog
django-admin startproject moj_projekt

# Wynik:
# moj_projekt/
#   └─ manage.py
#   └─ moj_projekt/
#       └─ __init__.py
#       └─ settings.py
#       └─ urls.py
#       └─ asgi.py
#       └─ wsgi.py

# Z kropką – tworzy pliki w bieżącym katalogu
django-admin startproject moj_projekt .

# Wynik (w bieżącym katalogu):
# └─ manage.py
# └─ moj_projekt/
#     └─ __init__.py
#     └─ settings.py
#     └─ urls.py
#     └─ asgi.py
#     └─ wsgi.py
```

Alternatywny sposób — przez manage.py:

```
# Jeśli masz już manage.py (np. z django-admin startproject)
python manage.py startproject moj_projekt .
```

Reguły nazewnictwa projektu:

- Używaj małych liter i podkreślników: `moj_projekt`, `sklep_internetowy`

- Unikaj nazw pokrywających się z modułami Pythona: nie używaj `test`, `django`, `site`
- Nazwa musi być poprawnym identyfikatorem Pythona (bez spacji, myślników, nie zaczyna się od cyfry)

Pełna procedura tworzenia nowego projektu Django od zera:

```
# 1. Utwórz katalog projektu
mkdir moj_projekt
cd moj_projekt

# 2. Utwórz wirtualne środowisko
python -m venv venv      # Windows
python3 -m venv venv     # Linux/macOS

# 3. Aktywuj środowisko
venv\Scripts\activate   # Windows (cmd)
source venv/bin/activate # Linux/macOS

# 4. Zainstaluj Django
pip install django

# 5. Utwórz projekt Django (z kropką!)
django-admin startproject config .

# 6. Sprawdź strukturę plików
dir                      # Windows
ls -la                   # Linux/macOS
```

Tip: Wielu programistów nazywa wewnętrzny katalog projektu `config` zamiast powtarzać nazwę projektu. Dzięki temu mamy jasną strukturę: `config/settings.py`, `config/urls.py`.

2.4. Struktura plików projektu (`manage.py`, `settings.py`, `urls.py`, `wsgi.py`, `asgi.py`)

Po utworzeniu projektu Django generuje kilka plików konfiguracyjnych. Zrozumienie ich roli jest kluczowe dla dalszej pracy.

Struktura plików projektu:

moj_projekt/	← Katalog główny projektu
├─ manage.py	← Narzędzie wiersza poleceń
├─ venv/	← Środowisko wirtualne (nie commitujemy!)
└─ config/	← Pakiet konfiguracyjny projektu
├─ __init__.py	← Oznacza katalog jako pakiet Pythona
├─ settings.py	← Główne ustawienia projektu
├─ urls.py	← Główna konfiguracja URL
├─ wsgi.py	← Punkt wejścia WSGI (serwer produkcyjny)
└─ asgi.py	← Punkt wejścia ASGI (serwer asynchroniczny)

Opis każdego pliku:

PLIK	ROLA	KIEDY MODYFIKUJESZ
<code>manage.py</code>	Narzędzie CLI do zarządzania projektem	Prawie nigdy
<code>__init__.py</code>	Oznacza katalog jako pakiet Pythona	Nigdy
<code>settings.py</code>	Wszystkie ustawienia projektu	Często
<code>urls.py</code>	Główna konfiguracja routingu URL	Przy dodawaniu nowych aplikacji
<code>wsgi.py</code>	Interfejs WSGI dla serwera produkcyjnego	Rzadko
<code>asgi.py</code>	Interfejs ASGI dla serwera asynchronicznego	Rzadko

manage.py — narzędzie wiersza poleceń:

Plik `manage.py` to skrypt Pythona, który jest głównym narzędziem do zarządzania projektem Django. Zamiast używać `django-admin`, w ramach projektu używamy `python manage.py <polecenie>`.

```
#!/usr/bin/env python

"""Django's command-line utility for administrative tasks."""
import os
import sys


def main():
    """Run administrative tasks."""
    # Ustawia zmienną środowiskową wskazującą na plik settings.py
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)


if __name__ == '__main__':
    main()
```

Najważniejsze polecenia manage.py:

POLECENIE	OPIS
<code>python manage.py runserver</code>	Uruchomienie serwera deweloperskiego
<code>python manage.py startapp <nazwa></code>	Tworzenie nowej aplikacji
<code>python manage.py makemigrations</code>	Tworzenie migracji
<code>python manage.py migrate</code>	Stosowanie migracji do bazy
<code>python manage.py createsuperuser</code>	Tworzenie konta administratora
<code>python manage.py shell</code>	Interaktywna konsola Pythona z Django
<code>python manage.py collectstatic</code>	Zbieranie plików statycznych
<code>python manage.py test</code>	Uruchamianie testów
<code>python manage.py check</code>	Sprawdzanie projektu pod kątem błędów
<code>python manage.py showmigrations</code>	Wyświetlenie stanu migracji

settings.py — główne ustawienia projektu:

Plik `settings.py` zawiera wszystkie ustawienia projektu Django. Poniżej pełny plik z komentarzami:

```
"""
Django settings for config project.
Plik konfiguracyjny projektu Django – zawiera wszystkie ustawienia.
"""

from pathlib import Path

# Ścieżka bazowa projektu – wszystkie inne ścieżki są względne do niej
# BASE_DIR wskazuje na katalog, w którym znajduje się manage.py
BASE_DIR = Path(__file__).resolve().parent.parent

# =====
# USTAWIENIA BEZPIECZEŃSTWA
# =====

# UWAGA: Klucz SECRET_KEY jest generowany automatycznie.
# W produkcji NIE trzymaj go w kodzie – użyj zmiennych środowiskowych!
SECRET_KEY = 'django-insecure-twoj-tajny-klucz-tutaj'

# UWAGA: DEBUG = True TYLKO w trakcie developmentu!
# W produkcji ZAWSZE ustaw DEBUG = False
DEBUG = True

# Lista domen/hostów, na których działa aplikacja
# W produkcji dodaj swoją domenę, np. ['mojadomena.pl', 'www.mojadomena.pl']
ALLOWED_HOSTS = []

# =====
# ZAINSTALOWANE APLIKACJE
# =====

INSTALLED_APPS = [
```

```

# Aplikacje wbudowane w Django
'django.contrib.admin',          # Panel administracyjny
'django.contrib.auth',          # System uwierzytelniania
'django.contrib.contenttypes',  # Framework typów zawartości
'django.contrib.sessions',      # Framework sesji
'django.contrib.messages',      # Framework wiadomości
'django.contrib.staticfiles',   # Serwowanie plików statycznych

# Aplikacje zewnętrzne (third-party) – tu dodajemy pakiety z pip
# np. 'rest_framework',
# np. 'crispy_forms',

# Nasze własne aplikacje – tu dodajemy nasze aplikacje
# np. 'blog.apps.BlogConfig',
# np. 'sklep.apps.SklepConfig',
]

# =====
# MIDDLEWARE
# =====

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',          # Bezpieczeństwo
    'django.contrib.sessions.middleware.SessionMiddleware',    # Sesje
    'django.middleware.common.CommonMiddleware',              # Wspólne operacje
    'django.middleware.csrf.CsrfViewMiddleware',              # Ochrona CSRF
    'django.contrib.auth.middleware.AuthenticationMiddleware',  # Uwierzytelnianie
    'django.contrib.messages.middleware.MessageMiddleware',    # Wiadomości
    'django.middleware.clickjacking.XFrameOptionsMiddleware',  # Ochrona clickjacking
]

# =====
# ROUTING URL
# =====

# Wskazuje na główny plik URL projektu
ROOT_URLCONF = 'config.urls'

# =====
# SZABLONY (TEMPLATES)
# =====

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        # DIRS – lista katalogów, w których Django szuka szablonów
        'DIRS': [BASE_DIR / 'templates'],
        # APP_DIRS = True – Django szuka szablonów w katalogach templates/
        # wewnątrz każdej zainstalowanej aplikacji
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',

```

```

        'django.contrib.auth.context_processors.auth',
        'django.contrib.messages.context_processors.messages',
    ],
},
],

# =====
# WSGI
# =====

WSGI_APPLICATION = 'config.wsgi.application'

# =====
# BAZA DANYCH
# =====

DATABASES = {
    'default': {
        # SQLite – domyślna baza, idealna do developmentu
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# =====
# WALIDACJA HASEŁ
# =====

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# =====
# INTERNACJONALIZACJA (i18n)
# =====

# Język domyślny – 'pl' dla polskiego
LANGUAGE_CODE = 'pl'

# Strefa czasowa – 'Europe/Warsaw' dla Polski
TIME_ZONE = 'Europe/Warsaw'

```

```
# Włączenie systemu tłumaczeń
USE_I18N = True

# Włączenie obsługi stref czasowych
USE_TZ = True

# =====
# PLIKI STATYCZNE (CSS, JavaScript, obrazy)
# =====

# URL, pod którym serwowane są pliki statyczne
STATIC_URL = 'static/'

# Katalogi z plikami statycznymi (poza aplikacjami)
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]

# Katalog, do którego collectstatic zbiera pliki (produkcja)
# STATIC_ROOT = BASE_DIR / 'staticfiles'

# =====
# DOMYŚLNE POLE KLUCZA GŁÓWNEGO
# =====

# Typ automatycznego klucza głównego (id) dla modeli
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

urls.py — konfiguracja routingu URL:

```
"""
URL configuration for config project.
Główny plik routingu URL – mapuje adresy URL na widoki.
"""

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    # Panel administracyjny – domyślnie pod /admin/
    path('admin/', admin.site.urls),

    # Tu będziemy dodawać URL-e naszych aplikacji, np.:
    # path('blog/', include('blog.urls')),
    # path('sklep/', include('sklep.urls')),
]
```

wsgi.py — interfejs WSGI:

```
"""
WSGI config for config project.
WSGI (Web Server Gateway Interface) – standardowy interfejs
```

```
między serwerem webowym a aplikacją Pythona.
Używany przez serwery produkcyjne: Gunicorn, uWSGI.
"""

import os
from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
application = get_wsgi_application()
```

asgi.py — interfejs ASGI:

```
"""
ASGI config for config project.
ASGI (Asynchronous Server Gateway Interface) – asynchroniczna
wersja WSGI. Obsługuje WebSocket, HTTP/2, Server-Sent Events.
Używany przez serwery: Daphne, Uvicorn, Hypercorn.
"""

import os
from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
application = get_asgi_application()
```

2.5. Uruchomienie serwera deweloperskiego — `python manage.py runserver`

Serwer deweloperski to wbudowany w Django lekki serwer HTTP przeznaczony do celów testowych i deweloperskich. **Nie należy** go używać w produkcji — nie jest zoptymalizowany pod kątem bezpieczeństwa ani wydajności. Do produkcji używa się serwerów takich jak Gunicorn, uWSGI czy Daphne.

Uruchomienie serwera:

```
# Domyślnie na porcie 8000
python manage.py runserver

# Na innym porcie (np. 8080)
python manage.py runserver 8080

# Na określonym adresie IP i porcie
python manage.py runserver 0.0.0.0:8000

# Dostęp z innych urządzeń w sieci lokalnej
python manage.py runserver 0.0.0.0:8000
```

Wynik w terminalu po uruchomieniu:

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
```

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.

May 30, 2026 - 19:00:00

Django version 5.2, using settings 'config.settings'

Starting development server at http://127.0.0.1:8000/

Quit the server with CONTROL-C.

Ważne informacje:

ELEMENT	OPIS
<code>http://127.0.0.1:8000/</code>	Adres serwera — otwórz go w przeglądarce
<code>CONTROL-C</code> (Ctrl+C)	Zatrzymanie serwera
<code>StatReloader</code>	Automatyczne przeładowanie po zmianach w kodzie
<code>18 unapplied migrations</code>	Ostrzeżenie — trzeba wykonać <code>python manage.py migrate</code>

Pierwszy krok po uruchomieniu — zastosowanie migracji:

```
# Zatrzymaj serwer (Ctrl+C), potem:
python manage.py migrate

# Wynik:
# Operations to perform:
#   Apply all migrations: admin, auth, contenttypes, sessions
# Running migrations:
#   Applying contenttypes.0001_initial... OK
#   Applying auth.0001_initial... OK
#   Applying admin.0001_initial... OK
#   Applying admin.0002_logentry_remove_auto_add... OK
#   ...
#   Applying sessions.0001_initial... OK

# Teraz uruchom serwer ponownie
python manage.py runserver
```

Po wejściu na `http://127.0.0.1:8000/` w przeglądarce zobaczysz stronę powitalną Django z komunikatem „The install worked successfully! Congratulations!”.

Tworzenie konta superużytkownika (administratora):

```
python manage.py createsuperuser

# Django zapyta o:
# Username (leave blank to use 'user'): admin
# Email address: admin@example.com
# Password: *****
# Password (again): *****
# Superuser created successfully.
```

Po utworzeniu superużytkownika możesz zalogować się do panelu administracyjnego pod adresem `http://127.0.0.1:8000/admin/`.

2.6. Tworzenie aplikacji — python manage.py startapp

W Django **projekt** to kontener konfiguracyjny, a **aplikacja** (app) to konkretny moduł funkcjonalności. Jeden projekt może zawierać wiele aplikacji. Każda aplikacja powinna realizować jedno, jasno określone zadanie (np. blog, sklep, konto użytkownika).

Tworzenie aplikacji:

```
# Składnia: python manage.py startapp <nazwa_aplikacji>
python manage.py startapp blog
```

Przykłady nazewnictwa aplikacji:

DOBRA NAZWA	ZŁA NAZWA	DLACZEGO?
<code>blog</code>	<code>Blog</code>	Nazwy aplikacji małymi literami
<code>sklep</code>	<code>moj-sklep</code>	Bez myślników — użyj podkreślnika
<code>konto_uzytkownika</code>	<code>KontoUzytkownika</code>	Bez camelCase
<code>produkty</code>	<code>test</code>	Nie koliduj z modułami Pythona
<code>zamowienia</code>	<code>1sklep</code>	Nie zaczynaj od cyfry

Struktura projektu z aplikacją:

```
moj_projekt/
├─ manage.py
├─ config/
│   ├─ __init__.py
│   ├─ settings.py
│   ├─ urls.py
│   ├─ wsgi.py
│   └─ asgi.py
├─ blog/                ← Nowa aplikacja
│   ├─ __init__.py
│   ├─ admin.py
│   ├─ apps.py
│   ├─ migrations/
│   │   └─ __init__.py
│   ├─ models.py
│   ├─ tests.py
│   └─ views.py
├─ templates/           ← Katalog szablonów (utworzymy ręcznie)
├─ static/              ← Katalog plików statycznych (utworzymy ręcznie)
└─ venv/               ← Środowisko wirtualne
```

Tworzenie wielu aplikacji:

```
python manage.py startapp blog
python manage.py startapp sklep
```



```
python manage.py startapp konto
python manage.py startapp api
```

Każda z tych aplikacji jest niezależnym modulem, który można włączyć lub wyłączyć w `INSTALLED_APPS`.

2.7. Rejestracja aplikacji w `INSTALLED_APPS`

Po utworzeniu aplikacji **musisz** ją zarejestrować w pliku `settings.py` w liście `INSTALLED_APPS`. Bez rejestracji Django nie będzie wiedziało o istnieniu aplikacji — nie będzie szukało jej modeli, szablonów, plików statycznych ani migracji.

Rejestracja aplikacji w `settings.py`:

```
# config/settings.py

INSTALLED_APPS = [

    # Wbudowane aplikacje Django

    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Aplikacje zewnętrzne (third-party)

    'rest_framework',
    'crispy_forms',

    # Nasze aplikacje – DODAJ TUTAJ!

    'blog.apps.BlogConfig',      # Zalecany sposób – wskazanie klasy AppConfig
    # lub krótsza forma:
    'blog',                     # Działa, ale mniej precyzyjne

]
```

Dwa sposoby rejestracji aplikacji:

SPOSÓB	PRZYKŁAD	OPIS
Pełna ścieżka do AppConfig	<code>'blog.apps.BlogConfig'</code>	Zalecany — wskazuje na klasę konfiguracyjną
Krótką nazwa	<code>'blog'</code>	Działa, ale Django musi sam znaleźć AppConfig

Klasa `AppConfig` — plik `apps.py`:

Każda aplikacja ma plik `apps.py` z klasą konfiguracyjną:

```
# blog/apps.py

from django.contrib import admin
from django.apps import AppConfig

class BlogConfig(AppConfig):
    """Klasa konfiguracyjna aplikacji blog."""

    # Typ domyślnego klucza głównego dla modeli tej aplikacji
```

```

default_auto_field = 'django.db.models.BigAutoField'

# Nazwa aplikacji (musi odpowiadać nazwie katalogu)
name = 'blog'

# Czytelna nazwa aplikacji (widoczna w panelu admina)
verbose_name = 'Blog'

def ready(self):
    """
    Metoda wywoływana po załadowaniu aplikacji.
    Tutaj można importować sygnały (signals).
    """
    # import blog.signals # odkomentuj, gdy dodasz sygnały
    pass

```

Sprawdzenie, czy aplikacja jest poprawnie zarejestrowana:

```

python manage.py check

# Wynik:
# System check identified no issues (0 silenced).

```

Jeśli aplikacja nie jest zarejestrowana, a próbujesz użyć jej modeli, Django zgłosi błąd.

2.8. Struktura plików aplikacji (models.py, views.py, urls.py, admin.py, apps.py, tests.py)

Każda aplikacja Django ma stałą strukturę plików. Niektóre pliki są generowane automatycznie przez `startapp`, inne tworzymy ręcznie (np. `urls.py`, `forms.py`).

Pliki generowane automatycznie przez `startapp`:

```

blog/
├── __init__.py      ← Oznacza katalog jako pakiet Pythona
├── admin.py         ← Rejestracja modeli w panelu admina
├── apps.py          ← Konfiguracja aplikacji (AppConfig)
├── migrations/      ← Katalog migracji bazy danych
│   └── __init__.py
├── models.py        ← Definicje modeli (tabel w bazie)
├── tests.py         ← Testy jednostkowe
└── views.py         ← Widoki (logika aplikacji)

```

Pliki tworzone ręcznie (często potrzebne):

```

blog/
├── ...              ← pliki generowane automatycznie
├── urls.py          ← Routing URL aplikacji (tworzymy ręcznie!)
├── forms.py         ← Formularze Django (tworzymy ręcznie)
├── serializers.py   ← Serializery (dla API REST – tworzymy ręcznie)
├── signals.py       ← Sygnały (tworzymy ręcznie)
└── context_processors.py ← Procesory kontekstu (tworzymy ręcznie)

```

```
├── templatetags/      ← Własne tagi i filtry szablonów
│   ├── __init__.py
│   └── blog_tags.py
├── templates/         ← Szablony HTML aplikacji
│   └── blog/
│       ├── lista.html
│       └── szczegoly.html
├── static/            ← Pliki statyczne aplikacji
│   ├── blog/
│   ├── css/
│   ├── js/
│   └── img/
└── management/        ← Własne komendy manage.py
    ├── commands/
    └── moja_komenda.py
```

Opis każdego pliku:

PLIK	ROLA	OPIS
<code>__init__.py</code>	Pakiet Pythona	Pusty plik oznaczający katalog jako pakiet
<code>models.py</code>	Modele danych	Definicje klas modeli (tabel w bazie danych)
<code>views.py</code>	Widoki	Funkcje/klasz obsługujące żądania HTTP
<code>urls.py</code>	Routing URL	Mapowanie adresów URL na widoki (tworzymy ręcznie!)
<code>admin.py</code>	Panel admina	Rejestracja modeli w panelu administracyjnym
<code>apps.py</code>	Konfiguracja	Klasa AppConfig z metadanymi aplikacji
<code>tests.py</code>	Testy	Testy jednostkowe i integracyjne
<code>forms.py</code>	Formularze	Klasz formularzy (tworzymy ręcznie)
<code>migrations/</code>	Migracje	Pliki migracji bazy danych (generowane automatycznie)

Przykład pełnej struktury aplikacji blog z zawartością każdego pliku:

```
# blog/models.py – definicje modeli
from django.db import models
from django.urls import reverse

class Kategoria(models.Model):
    """Model reprezentujący kategorię artykułów."""
    nazwa = models.CharField(max_length=100, unique=True, verbose_name="Nazwa")
    opis = models.TextField(blank=True, verbose_name="Opis")
    slug = models.SlugField(max_length=100, unique=True, verbose_name="Slug")

    class Meta:
        verbose_name = "Kategoria"
        verbose_name_plural = "Kategorie"
        ordering = ['nazwa']

    def __str__(self):
```

```

        return self.nazwa

    def get_absolute_url(self):
        return reverse('blog:kategoria', kwargs={'slug': self.slug})

class Post(models.Model):
    """Model reprezentujący post na blogu."""
    tytuł = models.CharField(max_length=200, verbose_name="Tytuł")
    slug = models.SlugField(max_length=200, unique=True, verbose_name="Slug")
    tresc = models.TextField(verbose_name="Treść")
    kategoria = models.ForeignKey(
        Kategoria,
        on_delete=models.CASCADE,
        related_name='posty',
        verbose_name="Kategoria"
    )
    opublikowany = models.BooleanField(default=False, verbose_name="Opublikowany")
    data_utworzenia = models.DateTimeField(auto_now_add=True, verbose_name="Data utworzenia")
    data_modyfikacji = models.DateTimeField(auto_now=True, verbose_name="Data modyfikacji")

    class Meta:
        verbose_name = "Post"
        verbose_name_plural = "Posty"
        ordering = ['-data_utworzenia']

    def __str__(self):
        return self.tytuł

    def get_absolute_url(self):
        return reverse('blog:szczegoly', kwargs={'slug': self.slug})

```

```

# blog/views.py – widoki

from django.shortcuts import render, get_object_or_404
from .models import Post, Kategoria

def lista_postow(request):
    """Widok listy wszystkich opublikowanych postów."""
    posty = Post.objects.filter(opublikowany=True)
    context = {
        'posty': posty,
        'tytul_strony': 'Blog – wszystkie posty',
    }
    return render(request, 'blog/lista.html', context)

def szczegoly_postu(request, slug):
    """Widok szczegółów pojedynczego posta."""
    post = get_object_or_404(Post, slug=slug, opublikowany=True)
    context = {
        'post': post,

```

```

    }

    return render(request, 'blog/szczegoly.html', context)

def posty_kategorii(request, slug):
    """Widok postów z wybranej kategorii."""
    kategoria = get_object_or_404(Kategoria, slug=slug)
    posty = Post.objects.filter(kategoria=kategoria, opublikowany=True)
    context = {
        'kategoria': kategoria,
        'posty': posty,
    }
    return render(request, 'blog/kategoria.html', context)

```

```

# blog/urls.py – routing URL aplikacji (tworzymy ręcznie!)
from django.urls import path
from . import views

# Przestrzeń nazw aplikacji – pozwala unikać kolizji nazw URL
app_name = 'blog'

urlpatterns = [
    # Przykład: /blog/
    path('', views.lista_postow, name='lista'),
    # Przykład: /blog/moj-pierwszy-post/
    path('<slug:slug>/', views.szczegoly_postu, name='szczegoly'),
    # Przykład: /blog/kategoria/python/
    path('kategoria/<slug:slug>/', views.posty_kategorii, name='kategoria'),
]

```

```

# blog/admin.py – rejestracja modeli w panelu administracyjnym
from django.contrib import admin
from .models import Kategoria, Post

@admin.register(Kategoria)
class KategoriaAdmin(admin.ModelAdmin):
    """Konfiguracja panelu admina dla kategorii."""
    list_display = ['nazwa', 'slug']
    prepopulated_fields = {'slug': ('nazwa',)}
    search_fields = ['nazwa']

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    """Konfiguracja panelu admina dla postów."""
    list_display = ['tytuł', 'kategoria', 'opublikowany', 'data_utworzenia']
    list_filter = ['opublikowany', 'kategoria', 'data_utworzenia']
    search_fields = ['tytuł', 'tresc']
    prepopulated_fields = {'slug': ('tytuł',)}

```

```
date_hierarchy = 'data_utworzenia'
ordering = ['-data_utworzenia']
list_editable = ['opublikowany']
```

```
# blog/apps.py – konfiguracja aplikacji
from django.apps import AppConfig

class BlogConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'blog'
    verbose_name = 'Blog'
```

```
# blog/tests.py – testy jednostkowe
from django.test import TestCase
from .models import Kategoria, Post

class KategoriaModelTest(TestCase):
    """Testy dla modelu Kategoria."""

    def setUp(self):
        """Przygotowanie danych testowych."""
        self.kategoria = Kategoria.objects.create(
            nazwa="Python",
            slug="python",
            opis="Artykuły o Pythonie"
        )

    def test_str(self):
        """Test metody __str__."""
        self.assertEqual(str(self.kategoria), "Python")

    def test_get_absolute_url(self):
        """Test metody get_absolute_url."""
        self.assertEqual(
            self.kategoria.get_absolute_url(),
            '/blog/kategoria/python/'
        )
```

Podłączenie URL-i aplikacji do głównego urls.py projektu:

```
# config/urls.py – główny plik URL projektu
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
```

```
path('blog/', include('blog.urls')), # Dołączenie URL-i aplikacji blog
]
```

3. Widoki (Views) — serce logiki aplikacji

Widoki to centralny element każdej aplikacji Django. To właśnie w widokach przetwarzamy żądania użytkowników, komunikujemy się z bazą danych, wykonujemy logikę biznesową i zwracamy odpowiedzi HTTP. W Django widok to po prostu funkcja Pythona (lub klasa) przyjmująca obiekt `HttpRequest` i zwracająca obiekt `HttpResponse`.

3.1. Czym jest widok — funkcja przyjmująca request, zwracająca response

Widok (view) w Django to **callable** (funkcja lub klasa) odpowiedzialny za przetwarzanie żądania HTTP i zwrócenie odpowiedzi HTTP. Każdy widok:

1. Przyjmuje jako pierwszy argument obiekt `HttpRequest` (zawierający informacje o żądaniu: metodę HTTP, nagłówki, dane formularza, ciasteczka, sesję itp.).
2. Zwraca obiekt `HttpResponse` (zawierający treść odpowiedzi: HTML, JSON, redirect, błąd itp.).

Obiekt `HttpRequest` — najważniejsze atrybuty:

ATRYBUT	TYP	OPIS
<code>request.method</code>	<code>str</code>	Metoda HTTP: <code>'GET'</code> , <code>'POST'</code> , <code>'PUT'</code> , <code>'DELETE'</code> itp.
<code>request.GET</code>	<code>QueryDict</code>	Parametry z query stringa (np. <code>?szukaj=django</code>)
<code>request.POST</code>	<code>QueryDict</code>	Dane z formularza wysłanego metodą POST
<code>request.FILES</code>	<code>MultiValueDict</code>	Przesłane pliki
<code>request.path</code>	<code>str</code>	Ścieżka URL (np. <code>/blog/artykuł/5/</code>)
<code>request.user</code>	<code>User</code>	Zalogowany użytkownik (lub <code>AnonymousUser</code>)
<code>request.session</code>	<code>dict-like</code>	Dane sesji użytkownika
<code>request.COOKIES</code>	<code>dict</code>	Ciasteczka (cookies)
<code>request.META</code>	<code>dict</code>	Nagłówki HTTP i metadane serwera
<code>request.content_type</code>	<code>str</code>	Typ zawartości żądania (np. <code>application/json</code>)
<code>request.body</code>	<code>bytes</code>	Surowe ciało żądania

Obiekt `HttpResponse` — tworzenie odpowiedzi:

```
from django.http import HttpResponse

# Odpowiedź tekstowa
response = HttpResponse("Witaj świecie!")

# Odpowiedź HTML
response = HttpResponse("<h1>Witaj!</h1>", content_type="text/html")

# Odpowiedź JSON
import json
response = HttpResponse(
```

```

    json.dumps({'status': 'ok', 'wiadomosc': 'Sukces'}),
    content_type="application/json"
)

# Ustawienie kodu statusu
response = HttpResponse("Nie znaleziono", status=404)

# Ustawienie nagłówka
response = HttpResponse("OK")
response['X-Custom-Header'] = 'wartość'

# Ustawienie ciasteczka
response.set_cookie('nazwa', 'wartość', max_age=3600)

```

Schemat działania widoku:



Prosty widok z pełnym wykorzystaniem request:

```

# views.py – widok demonstrujący atrybuty request
from django.http import HttpResponse

def informacje_o_zadaniu(request):
    """
    Widok wyświetlający informacje o żądaniu HTTP.
    Przydatny do debugowania.
    """
    info = []
    info.append(f"Metoda HTTP: {request.method}")
    info.append(f"Ścieżka: {request.path}")
    info.append(f"Pełny URL: {request.get_full_path()}")
    info.append(f"Użytkownik: {request.user}")

```



```
info.append(f"Czy zalogowany: {request.user.is_authenticated}")
info.append(f"IP klienta: {request.META.get('REMOTE_ADDR', 'nieznany')}")
info.append(f"Przeglądarka: {request.META.get('HTTP_USER_AGENT', 'nieznana')}")
info.append(f"Parametry GET: {dict(request.GET)}")
info.append(f"Content-Type: {request.content_type}")

tresc = "\n".join(info)

return HttpResponse(f"<pre>{tresc}</pre>", content_type="text/html")
```

3.2. Pierwszy widok — HttpResponse("Hello World")

Aby stworzyć swój pierwszy widok w Django, potrzebujemy dwóch rzeczy: funkcji widoku w `views.py` i wpisu w `urls.py` mapującego URL na ten widok.

Krok 1: Utworzenie widoku

```
# blog/views.py – pierwszy widok
from django.http import HttpResponse

def hello_world(request):
    """
    Najprostszy możliwy widok w Django.
    Przyjmuje żądanie HTTP i zwraca odpowiedź z tekstem.
    """
    return HttpResponse("Hello, World! Witaj w Django!")

def strona_glowna(request):
    """
    Widok strony głównej zwracający HTML.
    """
    html = """
<!DOCTYPE html>
<html lang="pl">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Moja strona Django</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 800px;
            margin: 50px auto;
            padding: 20px;
            background-color: #f5f5f5;
        }
        h1 { color: #092e20; }
        p { color: #333; line-height: 1.6; }
        .highlight { color: #092e20; font-weight: bold; }
    </style>
</head>
    """
```

```

<body>

    <h1>Witaj w Django!</h1>

    <p>To jest moja pierwsza strona napisana w
        <span class="highlight">Django</span>.</p>

    <p>Django to framework webowy napisany w Pythonie.</p>

    <ul>

        <li>Wersja Django: 5.2</li>

        <li>Język: Python 3.12</li>

        <li>Baza danych: SQLite</li>

    </ul>

</body>
</html>
"""

return HttpResponse(html)


def aktualna_data(request):
    """
    Widok wyświetlający aktualną datę i godzinę.
    Demonstruje użycie Pythona w widoku.
    """

    from datetime import datetime
    teraz = datetime.now()
    html = f"""
<html>
<body>

    <h1>Aktualna data i godzina</h1>

    <p>Data: {teraz.strftime('%d.%m.%Y')}</p>

    <p>Godzina: {teraz.strftime('%H:%M:%S')}</p>

    <p>Dzień tygodnia: {teraz.strftime('%A')}</p>

</body>
</html>
"""

    return HttpResponse(html)

```

Krok 2: Konfiguracja URL — mapowanie adresów na widoki

```

# blog/urls.py – routing URL aplikacji blog
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    # URL: /blog/
    path('', views.strona_glowna, name='strona_glowna'),
    # URL: /blog/hello/
    path('hello/', views.hello_world, name='hello'),
    # URL: /blog/data/
    path('data/', views.aktualna_data, name='data'),
]

```

Krok 3: Podłączenie URL-i aplikacji do projektu

```
# config/urls.py – główny routing URL projektu

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('blog/', include('blog.urls')), # Dołączenie URL-i z aplikacji blog
]
```

Teraz możesz otworzyć w przeglądarce:

- `http://127.0.0.1:8000/blog/` → strona główna bloga
- `http://127.0.0.1:8000/blog/hello/` → Hello World
- `http://127.0.0.1:8000/blog/data/` → aktualna data

3.3. Routing URL — `path()`, `include()`, `patterns`

System routingu URL w Django odpowiada za mapowanie adresów URL na odpowiednie widoki. Gdy użytkownik wpisuje adres w przeglądarce, Django przechodzi przez listę wzorców URL (`urlpatterns`) i szuka pierwszego pasującego wzorca.

Funkcja `path()` — podstawowy sposób definiowania URL:

```
from django.urls import path

# Składnia: path(route, view, kwargs=None, name=None)
path('artykuły/', views.lista_artykułow, name='lista_artykułow')
```

ARGUMENT	OPIS
<code>route</code>	Wzorzec URL (string), np. <code>'artykuły/'</code>
<code>view</code>	Funkcja widoku lub klasa widoku
<code>kwargs</code>	Opcjonalne dodatkowe argumenty przekazywane do widoku
<code>name</code>	Opcjonalna nazwa URL (do odwoływania się w szablonach i kodzie)

Funkcja `include()` — dołączanie URL-i z aplikacji:

```
from django.urls import path, include

# Dołącza wszystkie URL-e z pliku blog/urls.py
# pod prefiksem /blog/
path('blog/', include('blog.urls'))

# Dołącza URL-e z przestrzenią nazw
path('blog/', include('blog.urls', namespace='blog'))

# Dołącza URL-e z wielu aplikacji
urlpatterns = [
    path('admin/', admin.site.urls),
```

```
path('blog/', include('blog.urls')),
path('sklep/', include('sklep.urls')),
path('api/', include('api.urls')),
path('konto/', include('konto.urls')),

]
```

Kompletny przykład routingu z wieloma aplikacjami:

```
# config/urls.py – główny plik URL projektu
from django.contrib import admin
from django.urls import path, include
from django.http import HttpResponse

def strona_glowna(request):
    """Widok strony głównej projektu (nie aplikacji)."""
    return HttpResponse("<h1>Strona główna</h1><ul>"
                        "<li><a href='/blog/'>Blog</a></li>"
                        "<li><a href='/admin/'>Admin</a></li></ul>")

urlpatterns = [
    # Strona główna – / (bez prefiksu)
    path('', strona_glowna, name='strona_glowna'),

    # Panel admina – /admin/
    path('admin/', admin.site.urls),

    # Aplikacja blog – wszystkie URL-e z blog/urls.py pod /blog/
    path('blog/', include('blog.urls')),

    # Aplikacja sklep – /sklep/
    # path('sklep/', include('sklep.urls')),
]
```

Kolejność dopasowania URL-i:

Django sprawdza wzorce URL **od góry do dołu**. Pierwszy pasujący wzorec jest używany. Dlatego kolejność ma znaczenie:

```
# blog/urls.py – kolejność ma znaczenie!
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    # UWAGA: kolejność od najbardziej specyficznego do najbardziej ogólnego

    # /blog/nowy/ – dopasuje się do tego wzorca
    path('nowy/', views.nowy_post, name='nowy'),

    # /blog/popularne/ – dopasuje się do tego wzorca
]
```

```

path('popularne/', views.popularne, name='popularne'),

# /blog/<slug>/ – dopasuje się do KAŻDEGO tekstu po /blog/
# Dlatego musi być NA KOŃCU!
path('<slug:slug>/', views.szczegoly_postu, name='szczegoly'),
]

```

Funkcja `re_path()` — zaawansowane wzorce z wyrażeniami regularnymi:

```

from django.urls import re_path
from . import views

urlpatterns = [
    # Wyrażenie regularne – rok w formacie 4 cyfr
    re_path(r'^artykuły/(?P<rok>d{4})/$', views.artykuły_rok, name='artykuły_rok'),

    # Rok i miesiąc
    re_path(
        r'^artykuły/(?P<rok>d{4})/(?P<miesiac>d{2})/$',
        views.artykuły_miesiac,
        name='artykuły_miesiac'
    ),
]

```

3.4. Parametry w URL — konwertery `int:pk`, `str:slug`, `uuid:id`

Django pozwala na przechwytywanie fragmentów URL jako parametrów, które są przekazywane do widoku. Używamy do tego **konwerterów ścieżek** w nawiasach ostrych `< >`.

Składnia: `<konwerter:nazwa_parametru>`

Dostępne konwertery ścieżek:

KONWERTER	OPIS	DOPASOWUJE	PRZYKŁAD URL
<code>str</code>	Dowolny tekst (bez <code>/</code>)	<code>[^/]+</code>	<code>/artykuł/hello-world/</code>
<code>int</code>	Liczba całkowita ≥ 0	<code>[0-9]+</code>	<code>/artykuł/42/</code>
<code>slug</code>	Slug (litery, cyfry, <code>-</code> , <code>_</code>)	<code>[-a-zA-Z0-9_]+</code>	<code>/artykuł/moj-post/</code>
<code>uuid</code>	UUID (format ze znakami <code>-</code>)	<code>[0-9a-f-]{36}</code>	<code>/uzytkownik/550e8400-e29b- ...</code>
<code>path</code>	Dowolny tekst (włącznie z <code>/</code>)	<code>.+</code>	<code>/pliki/sciezka/do/pliku.txt</code>

Przykłady użycia konwerterów:

```

# blog/urls.py – parametry w URL
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    # <int:pk> – liczba całkowita, np. /blog/artykuł/42/

```

```

path('artykul/<int:pk>/', views.artykul_szczegoly, name='artykul_szczegoly'),

# <str:slug> – slug, np. /blog/post/moj-pierwszy-post/
path('post/<slug:slug>/', views.post_szczegoly, name='post_szczegoly'),

# <uuid:id> – UUID, np. /blog/dokument/550e8400-e29b-41d4-a716-446655440000/
path('dokument/<uuid:id>/', views.dokument_szczegoly, name='dokument_szczegoly'),

# Wiele parametrów – /blog/archiwum/2026/05/
path('archiwum/<int:rok>/<int:miesiac>/', views.archiwum, name='archiwum'),

# <path:sciezka> – ścieżka z ukośnikami, np. /blog/pliki/css/style.css
path('pliki/<path:sciezka>/', views.pobierz_plik, name='pobierz_plik'),

# <str:nazwa> – domyślny konwerter str, np. /blog/autor/jan-kowalski/
path('autor/<str:nazwa>/', views.profil_autora, name='profil_autora'),

]

```

Widoki odbierające parametry z URL:

```

# blog/views.py – widoki z parametrami URL
from django.http import HttpResponse
from django.shortcuts import render, get_object_or_404
from .models import Post

def artykul_szczegoly(request, pk):
    """
    Widok szczegółów artykułu.
    Parametr 'pk' pochodzi z URL: /blog/artykul/<int:pk>/
    Django automatycznie konwertuje go na int.
    """
    post = get_object_or_404(Post, pk=pk)
    return render(request, 'blog/szczegoly.html', {'post': post})

def post_szczegoly(request, slug):
    """
    Widok szczegółów posta po slug.
    Parametr 'slug' pochodzi z URL: /blog/post/<slug:slug>/
    """
    post = get_object_or_404(Post, slug=slug, opublikowany=True)
    return render(request, 'blog/szczegoly.html', {'post': post})

def dokument_szczegoly(request, id):
    """
    Widok szczegółów dokumentu po UUID.
    Parametr 'id' pochodzi z URL: /blog/dokument/<uuid:id>/
    Django automatycznie konwertuje go na obiekt UUID.
    """
    return HttpResponse(f"Dokument o UUID: {id}")

```

```
def archiwum(request, rok, miesiac):
    """
    Widok archiwum – filtrowanie po roku i miesiącu.
    Parametry 'rok' i 'miesiac' pochodzą z URL: /blog/archiwum/<int:rok>/<int:miesiac>/
    """
    posty = Post.objects.filter(
        data_utworzenia__year=rok,
        data_utworzenia__month=miesiac,
        opublikowany=True
    )
    context = {
        'posty': posty,
        'rok': rok,
        'miesiac': miesiac,
    }
    return render(request, 'blog/archiwum.html', context)

def pobierz_plik(request, sciezka):
    """
    Widok obsługujący ścieżkę z ukośnikami.
    Konwerter 'path' pozwala na ukośniki w parametrze.
    """
    return HttpResponse(f"Żądany plik: {sciezka}")

def profil_autora(request, nazwa):
    """
    Widok profilu autora – parametr str (domyślny).
    """
    return HttpResponse(f"Profil autora: {nazwa}")
```

3.5. Nazwy URL — name='nazwa' i reverse()

Nazwy URL to mechanizm pozwalający odwoływać się do adresów URL za pomocą nazw symbolicznych, zamiast wpisywać adresy na sztywno w kodzie. Dzięki temu, gdy zmienisz adres URL w `urls.py`, nie musisz zmieniać go w szablonach i widokach — wystarczy odwołanie do nazwy.

Definiowanie nazw URL:

```
# blog/urls.py – każdy path() ma name='...'
from django.urls import path
from . import views

app_name = 'blog' # Przestrzeń nazw (namespace) – ważne dla unikania kolizji!

urlpatterns = [
    path('', views.lista_postow, name='lista'),
    path('<int:pk>/', views.szczegoly_postu, name='szczegoly'),
    path('nowy/', views.nowy_post, name='nowy'),
    path('<int:pk>/edytuj/', views.edytuj_post, name='edytuj'),
```

```
path('<int:pk>/usun/', views.usun_post, name='usun'),
path('kategoria/<slug:slug>/', views.posty_kategorii, name='kategoria'),
]
```

Odwoływanie się do nazw URL w szablonach:

```
←!— Użycie tagu {% url %} w szablonach →

←!— URL bez parametrów: /blog/ →
<a href="{% url 'blog:lista' %}">Lista postów</a>

←!— URL z parametrem int: /blog/42/ →
<a href="{% url 'blog:szczegoly' pk=post.pk %}">{{ post.tytul }}</a>

←!— URL z parametrem slug: /blog/kategoria/python/ →
<a href="{% url 'blog:kategoria' slug=kategoria.slug %}">{{ kategoria.nazwa }}</a>

←!— URL do panelu admina →
<a href="{% url 'admin:index' %}">Panel admina</a>

←!— URL z wieloma parametrami →
<a href="{% url 'blog:archiwum' rok=2026 miesiac=5 %}">Maj 2026</a>
```

Funkcja `reverse()` — odwoływanie się do URL w kodzie Pythona:

```
# views.py – użycie reverse() i redirect()
from django.urls import reverse
from django.shortcuts import redirect

def po_zapisaniu(request):
    """Przykład użycia reverse() do generowania URL-a."""

    # Generowanie URL bez parametrów
    url = reverse('blog:lista')
    # Wynik: '/blog/'

    # Generowanie URL z parametrem
    url = reverse('blog:szczegoly', kwargs={'pk': 42})
    # Wynik: '/blog/42/'

    # Generowanie URL z parametrem slug
    url = reverse('blog:kategoria', kwargs={'slug': 'python'})
    # Wynik: '/blog/kategoria/python/'

    # Generowanie URL z argumentami pozycyjnymi
    url = reverse('blog:szczegoly', args=[42])
    # Wynik: '/blog/42/'

    return redirect(url)
```



```
def nowy_post(request):
    """Przykład przekierowania po zapisaniu posta."""
    if request.method == 'POST':
        # ... logika zapisu posta ...
        # Po zapisaniu przekieruj na stronę szczegółów
        return redirect(reverse('blog:szczegoly', kwargs={'pk': 1}))

    return render(request, 'blog/nowy.html')
```

Przestrzenie nazw (namespaces):

Przestrzenie nazw zapobiegają kolizjom nazw URL między aplikacjami. Jeśli masz dwie aplikacje z URL-em o nazwie `lista`, przestrzeń nazw pozwala je rozróżnić:

```
# blog/urls.py
app_name = 'blog'
urlpatterns = [
    path('', views.lista_postow, name='lista'), # Pełna nazwa: 'blog:lista'
]

# sklep/urls.py
app_name = 'sklep'
urlpatterns = [
    path('', views.lista_produkow, name='lista'), # Pełna nazwa: 'sklep:lista'
]
```

```
←!— W szablonie — bez kolizji dzięki namespace →
<a href="{% url 'blog:lista' %}">Blog</a>
<a href="{% url 'sklep:lista' %}">Sklep</a>
```

3.6. Przekierowania — `redirect()`, `reverse()`

Przekierowania (redirects) to mechanizm wysyłania użytkownika pod inny adres URL. Są powszechnie używane po zapisaniu formularza (wzorec POST/Redirect/GET), po zalogowaniu, po usunięciu obiektu itp.

Funkcja `redirect()` — najczęściej używana:

```
from django.shortcuts import redirect

def widok_przekierowania(request):
    """Różne sposoby użycia redirect()."""

    # 1. Przekierowanie na nazwę URL
    return redirect('blog:lista')

    # 2. Przekierowanie na nazwę URL z parametrami
    return redirect('blog:szczegoly', pk=42)
```

```
# 3. Przekierowanie na pełny URL (string)
return redirect('/blog/')

# 4. Przekierowanie na zewnętrzny URL
return redirect('https://www.djangoproject.com/')

# 5. Przekierowanie na obiekt z metodą get_absolute_url()
post = Post.objects.get(pk=1)
return redirect(post) # Wywoła post.get_absolute_url()
```

Wzorzec POST/Redirect/GET (PRG):

Jest to standardowy wzorzec używany przy obsłudze formularzy. Zapobiega duplikowaniu danych przy odświeżeniu strony po wysłaniu formularza.

```
# views.py – wzorzec PRG (Post/Redirect/Get)
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from .models import Post

def edytuj_post(request, pk):
    """
    Widok edycji posta – wzorzec POST/Redirect/GET.
    """
    post = get_object_or_404(Post, pk=pk)

    if request.method == 'POST':
        # Przetwarzanie danych z formularza
        post.tytul = request.POST.get('tytul', post.tytul)
        post.tresc = request.POST.get('tresc', post.tresc)
        post.save()

        # Dodanie komunikatu sukcesu
        messages.success(request, 'Post został zaktualizowany!')

        # REDIRECT – przekierowanie na stronę szczegółów (GET)
        return redirect('blog:szczegoly', pk=post.pk)

    # GET – wyświetlenie formularza
    context = {'post': post}
    return render(request, 'blog/edytuj.html', context)

def usun_post(request, pk):
    """
    Widok usuwania posta – po usunięciu przekieruj na listę.
    """
    post = get_object_or_404(Post, pk=pk)

    if request.method == 'POST':
        post.delete()
        messages.success(request, 'Post został usunięty!')
```

```
return redirect('blog:lista') # Przekierowanie na listę

context = {'post': post}
return render(request, 'blog/potwierdz_usuniecie.html', context)
```

Typy przekierowań HTTP:

TYP	KOD HTTP	KLASA DJANGO	OPIS
Tymczasowe	302	HttpResponseRedirect	Domyślne — redirect() zwraca 302
Trwałe	301	HttpResponsePermanentRedirect	Trwałe przeniesienie (SEO)

```
from django.http import HttpResponseRedirect, HttpResponsePermanentRedirect
from django.shortcuts import redirect

# Przekierowanie tymczasowe (302) – domyślne
redirect('blog:lista') # Zwraca 302

# Przekierowanie trwałe (301)
redirect('blog:lista', permanent=True) # Zwraca 301

# Ręczne tworzenie przekierowania
HttpResponseRedirect('/blog/') # 302
HttpResponsePermanentRedirect('/blog/') # 301
```

3.7. Kody odpowiedzi HTTP — 200, 301, 302, 404, 500

Django pozwala na zwracanie odpowiedzi HTTP z różnymi kodami statusu. Znajomość kodów HTTP jest niezbędna do prawidłowego działania aplikacji webowej — kody informują przeglądarkę (i wyszukiwarki) o tym, co się stało z żądaniem.

Najczęstsze kody HTTP:

KOD	NAZWA	OPIS	UŻYCIE W DJANGO
200	OK	Żądanie zakończone sukcesem	HttpResponse() — domyślny
201	Created	Zasób został utworzony	API REST — po POST
204	No Content	Sukces, ale brak treści	API REST — po DELETE
301	Moved Permanently	Trwałe przekierowanie	redirect(url, permanent=True)
302	Found	Tymczasowe przekierowanie	redirect(url)
400	Bad Request	Błędne żądanie	HttpResponseBadRequest()
403	Forbidden	Brak uprawnień	HttpResponseForbidden()
404	Not Found	Strona nie znaleziona	Http404 / get_object_or_404()
405	Method Not Allowed	Metoda niedozwolona	HttpResponseNotAllowed()
500	Internal Server Error	Błąd serwera	Nieobsłużony wyjątek

Zwracanie różnych kodów w widokach:

```
# views.py – przykłady różnych kodów odpowiedzi HTTP

from django.http import (
    HttpResponse,
    HttpResponseRedirect,
    HttpResponsePermanentRedirect,
    HttpResponseBadRequest,
    HttpResponseForbidden,
    HttpResponseNotFound,
    HttpResponseNotAllowed,
    HttpResponseServerError,
    JsonResponse,
    Http404,
)

from django.shortcuts import get_object_or_404, render


# === 200 OK – domyślna odpowiedź ===
def widok_200(request):
    return HttpResponse("Wszystko OK!") # status=200 domyślnie


# === 201 Created – zasób utworzony (API) ===
def widok_201(request):
    return HttpResponse("Zasób utworzony", status=201)


# === 301 Moved Permanently – trwałe przekierowanie ===
def widok_301(request):
    return HttpResponsePermanentRedirect('/nowy-adres/')


# === 302 Found – tymczasowe przekierowanie ===
def widok_302(request):
    return HttpResponseRedirect('/tymczasowy-adres/')


# === 400 Bad Request – błędne żądanie ===
def widok_400(request):
    return HttpResponseBadRequest("Nieprawidłowe dane w żądaniu!")


# === 403 Forbidden – brak uprawnień ===
def widok_403(request):
    return HttpResponseForbidden("Nie masz uprawnień do tego zasobu!")


# === 404 Not Found – nie znaleziono ===
def widok_404_reczny(request):
    return HttpResponseNotFound("Strona nie została znaleziona.")


def widok_404_wyjatek(request, pk):
    """Preferowany sposób – rzucenie wyjątku Http404."""
```

```

from django.http import Http404
from .models import Post

try:
    post = Post.objects.get(pk=pk)
except Post.DoesNotExist:
    raise Http404("Post o podanym ID nie istnieje.")

return render(request, 'blog/szczegoly.html', {'post': post})

def widok_404_skrot(request, pk):
    """Najkrótszy sposób – get_object_or_404()."""
    from .models import Post
    post = get_object_or_404(Post, pk=pk)
    return render(request, 'blog/szczegoly.html', {'post': post})

# === 405 Method Not Allowed – metoda niedozwolona ===
def widok_405(request):
    if request.method != 'POST':
        return HttpResponseNotAllowed(['POST'], "Dozwolone tylko ządania POST!")
    return HttpResponse("OK")

# === 500 Internal Server Error – bład serwera ===
def widok_500(request):
    return HttpResponseServerError("Wystąpił bład serwera!")

# === JsonResponse – odpowiedź JSON (dla API) ===
def widok_json(request):
    dane = {
        'status': 'sukces',
        'wiadomosc': 'Dane pobrane poprawnie',
        'dane': [
            {'id': 1, 'nazwa': 'Produkt 1'},
            {'id': 2, 'nazwa': 'Produkt 2'},
        ]
    }
    return JsonResponse(dane, json_dumps_params={'ensure_ascii': False})

```

Własne strony błędów (404, 500):

Aby Django wyświetlało niestandardowe strony błędów, musisz:

1. Ustawić `DEBUG = False` w `settings.py`.
2. Dodać `ALLOWED_HOSTS` (np. `['localhost', '127.0.0.1']`).
3. Utworzyć szablony `404.html` i `500.html` w głównym katalogu `templates/`.

```

<!-- templates/404.html – niestandardowa strona 404 -->
<!DOCTYPE html>
<html lang="pl">

```

```

<head>

  <meta charset="UTF-8">

  <title>404 – Nie znaleziono</title>

  <style>

    body { font-family: Arial, sans-serif; text-align: center; padding: 100px; }

    h1 { font-size: 72px; color: #e74c3c; }

    p { font-size: 18px; color: #666; }

  </style>

</head>

<body>

  <h1>404</h1>

  <p>Strona, której szukasz, nie istnieje.</p>

  <a href="/">Wróć na stronę główną</a>

</body>

</html>

```

```

<!-- templates/500.html – niestandardowa strona 500 -->

<!DOCTYPE html>

<html lang="pl">

<head>

  <meta charset="UTF-8">

  <title>500 – Błąd serwera</title>

  <style>

    body { font-family: Arial, sans-serif; text-align: center; padding: 100px; }

    h1 { font-size: 72px; color: #e74c3c; }

    p { font-size: 18px; color: #666; }

  </style>

</head>

<body>

  <h1>500</h1>

  <p>Przepraszamy, wystąpił błąd serwera. Pracujemy nad rozwiązaniem problemu.</p>

  <a href="/">Wróć na stronę główną</a>

</body>

</html>

```

Konfiguracja własnych widoków błędów w urls.py:

```

# config/urls.py

from django.conf.urls import handler404, handler500

# Wskazanie własnych widoków obsługi błędów

handler404 = 'blog.views.custom_404'

handler500 = 'blog.views.custom_500'

```

```

# blog/views.py – własne widoki błędów

def custom_404(request, exception):

    """Własna strona błędu 404."""

    return render(request, '404.html', status=404)

```

```
def custom_500(request):
    """Własna strona błędu 500."""
    return render(request, '500.html', status=500)
```

3.8. Widoki oparte na klasach (CBV) vs widoki funkcyjne (FBV) — porównanie

Django oferuje dwa podejścia do tworzenia widoków: **widoki funkcyjne (FBV — Function-Based Views)** i **widoki oparte na klasach (CBV — Class-Based Views)**. Każde z nich ma swoje zalety i wady.

Widoki funkcyjne (FBV) — prostota i czytelność:

```
# views.py – widoki funkcyjne (FBV)
from django.shortcuts import render, get_object_or_404, redirect
from django.http import HttpResponseRedirect
from .models import Post

def lista_postow(request):
    """FBV – lista wszystkich postów."""
    posty = Post.objects.filter(opublikowany=True)
    return render(request, 'blog/lista.html', {'posty': posty})

def szczegoly_postu(request, pk):
    """FBV – szczegóły jednego posta."""
    post = get_object_or_404(Post, pk=pk)
    return render(request, 'blog/szczegoly.html', {'post': post})

def nowy_post(request):
    """FBV – tworzenie nowego posta (obsługa GET i POST)."""
    if request.method == 'POST':
        tytul = request.POST.get('tytul')
        tresc = request.POST.get('tresc')
        post = Post.objects.create(tytul=tytul, tresc=tresc)
        return redirect('blog:szczegoly', pk=post.pk)
    return render(request, 'blog/nowy.html')

def edytuj_post(request, pk):
    """FBV – edycja posta."""
    post = get_object_or_404(Post, pk=pk)
    if request.method == 'POST':
        post.tytul = request.POST.get('tytul', post.tytul)
        post.tresc = request.POST.get('tresc', post.tresc)
        post.save()
        return redirect('blog:szczegoly', pk=post.pk)
    return render(request, 'blog/edytuj.html', {'post': post})

def usun_post(request, pk):
```

```

"""FBV – usuwanie posta."""
post = get_object_or_404(Post, pk=pk)

if request.method == 'POST':
    post.delete()
    return redirect('blog:lista')
return render(request, 'blog/potwierdz_usuniecie.html', {'post': post})

```

Widoki oparte na klasach (CBV) — reużywalność i organizacja:

```

# views.py – widoki oparte na klasach (CBV)
from django.views.generic import (
    ListView,
    DetailView,
    CreateView,
    UpdateView,
    DeleteView,
)
from django.urls import reverse_lazy
from .models import Post

class ListaPostowView(ListView):
    """CBV – lista wszystkich postów."""
    model = Post
    template_name = 'blog/lista.html'
    context_object_name = 'posty'
    queryset = Post.objects.filter(opublikowany=True)
    paginate_by = 10 # Paginacja – 10 postów na stronę
    ordering = ['-data_utworzenia']

class SzczegolyPostuView(DetailView):
    """CBV – szczegóły jednego posta."""
    model = Post
    template_name = 'blog/szczegoly.html'
    context_object_name = 'post'

    def get_queryset(self):
        """Pokaż tylko opublikowane posty."""
        return Post.objects.filter(opublikowany=True)

class NowyPostView(CreateView):
    """CBV – tworzenie nowego posta."""
    model = Post
    template_name = 'blog/nowy.html'
    fields = ['tytul', 'tresc', 'kategoria', 'opublikowany']
    success_url = reverse_lazy('blog:lista')

    def form_valid(self, form):
        """Dodatkowa logika po walidacji formularza."""
        form.instance.author = self.request.user
        return super().form_valid(form)

```



```

class EdytujPostView(UpdateView):
    """CBV – edycja posta."""
    model = Post
    template_name = 'blog/edytuj.html'
    fields = ['tytuł', 'tresc', 'kategoria', 'opublikowany']
    context_object_name = 'post'

    def get_success_url(self):
        """Po edycji przekieruj na stronę szczegółów."""
        return reverse_lazy('blog:szczegoly', kwargs={'pk': self.object.pk})

class UsunPostView(DeleteView):
    """CBV – usuwanie posta."""
    model = Post
    template_name = 'blog/potwierdz_usuniecie.html'
    context_object_name = 'post'
    success_url = reverse_lazy('blog:lista')

```

Konfiguracja URL dla CBV:

```

# blog/urls.py – URL-e z widokami CBV
from django.urls import path
from . import views

app_name = 'blog'

urlpatterns = [
    # CBV wymaga wywołania .as_view()
    path('', views.ListaPostowView.as_view(), name='lista'),
    path('<int:pk>/', views.SzczegolyPostuView.as_view(), name='szczegoly'),
    path('nowy/', views.NowyPostView.as_view(), name='nowy'),
    path('<int:pk>/edytuj/', views.EdytujPostView.as_view(), name='edytuj'),
    path('<int:pk>/usun/', views.UsunPostView.as_view(), name='usun'),
]

```

Porównanie FBV vs CBV:

CECHA	FBV (FUNKCYJNE)	CBV (KLASOWE)
Czytelność	Bardzo czytelne, liniowe	Wymaga znajomości dziedziczenia
Prostota	Proste, jawne	Abstrakcyjne, dużo „magii”
Reużywalność	Trzeba kopiować logikę	Łatwe dziedziczenie i mixiny
Rozszerzalność	Dekoratory	Nadpisywanie metod, mixiny
Obsługa metod HTTP	<code>if request.method == 'POST':</code>	Osobne metody: <code>get()</code> , <code>post()</code>
Generyczne widoki	Brak	ListView, DetailView, CreateView...
Paginacja	Ręczna implementacja	<code>paginate_by = 10</code>
Kiedy używać	Proste widoki, logika niestandardowa	CRUD, standardowe operacje
Krzywa uczenia	Niska	Średnia/Wysoka
Testowanie	Proste	Nieco trudniejsze

Kiedy używać FBV, a kiedy CBV?

- **Używaj FBV** gdy: widok jest prosty, logika jest niestandardowa, dopiero uczysz się Django, potrzebujesz pełnej kontroli nad przepływem.
- **Używaj CBV** gdy: tworzysz standardowe operacje CRUD (lista, szczegóły, tworzenie, edycja, usuwanie), chcesz reużywać logikę, projekt jest duży i wymaga organizacji.

Przykład CBV z ręczną obsługą metod HTTP:

```
# views.py – CBV z ręcznym get() i post()
from django.views import View
from django.shortcuts import render, redirect, get_object_or_404
from .models import Post

class KontaktView(View):
    """
    CBV z ręczną obsługą metod HTTP.
    Klasa bazowa View daje pełną kontrolę nad obsługą GET i POST.
    """
    template_name = 'blog/kontakt.html'

    def get(self, request):
        """Obsługa żądania GET – wyświetlenie formularza."""
        return render(request, self.template_name)

    def post(self, request):
        """Obsługa żądania POST – przetworzenie formularza."""
        imie = request.POST.get('imie')
        email = request.POST.get('email')
        wiadomosc = request.POST.get('wiadomosc')

        # Tu logika wysyłania emaila, zapisu do bazy itp.
        # ...

        return redirect('blog:lista')
```

4. Szablony (Templates) — warstwa prezentacji

4.1. Konfiguracja szablonów

Aby Django widziało nasze pliki HTML, musimy je umieścić w specjalnym katalogu i powiedzieć frameworkowi, by tam szukał. Najlepszą praktyką jest stworzenie jednego wspólnego folderu `templates` w katalogu głównym projektu.

`mojprojekt/settings.py`:

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        # Dodajemy ścieżkę do wspólnego folderu szablonów!
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        'APP_DIRS': True, # Pozwala szukać też w folderze 'templates' wewnątrz każdej aplikacji
        'OPTIONS': {
            'context_processors': [
                # ...
            ],
        },
    ],
]
```

4.2. Renderowanie szablonu

Zamiast zwracać surowy `HttpResponse("tekst")`, możemy wygenerować odpowiedź z pliku HTML, wstrzykując do niego zmienne (tzw. `context`). Używamy do tego wbudowanej funkcji `render()`.

`blog/views.py`:

```
from django.shortcuts import render

def profil_uzytkownika(request):
    # Dane, które chcemy przekazać do HTML-a:
    dane = {
        'imie': 'Jan',
        'wiek': 25,
        'czy_aktywny': True
    }

    # render() przyjmuje: request, nazwe_pliku_html, słownik_ze_zmiennymi
    return render(request, 'profil.html', dane)
```

4.3. Język szablonów Django (DTL) — zmienne

Pliki `.html` w Django to nie jest zwykły HTML. Django nakłada na nie silnik szablonów, który wyłapuje specjalne znaczniki.

Aby wyświetlić zmienną przekazaną ze słownika `context`, używamy podwójnych nawiasów klamrowych `{{ zmienna }}`.

`templates/profil.html`:

```

<!DOCTYPE html>

<html>

<body>

    <h1>Profil użytkownika</h1>

    <p>Imię: {{ imie }}</p>

    <p>Wiek: {{ wiek }}</p>

</body>

</html>

```

Możemy też odwoływać się do atrybutów obiektów za pomocą KROPKI (bez nawiasów dla metod!):

```

{{ osoba.imie }}
{{ lista_produktow.0 }} ←— Indeksowanie listy →
{{ slownik.klucz }}

```

4.4. Tagi szablonów

Tagi to instrukcje programistyczne zapisywane wewnątrz `{% ... %}`. Służą m.in. do pętli i instrukcji warunkowych.

Instrukcja if:

```

{% if czy_aktywny %}

    <p>Użytkownik jest online.</p>

{% elif wiek > 18 %}

    <p>Użytkownik pełnoletni, ale offline.</p>

{% else %}

    <p>Brak dostępu.</p>

{% endif %}

```

Pętla for:

```

<ul>

{% for produkt in produkty %}

    <li>{{ produkt.nazwa }} - {{ produkt.cena }} zł</li>

{% empty %}

    <li>Lista produktów jest pusta.</li> ←— Działa jeśli lista nie istnieje lub jest pusta! →

{% endfor %}

</ul>

```

(Zauważ niezwykle przydatny tag `{% empty %} !`)

4.5. Filtry szablonów

Filtry służą do modyfikowania wartości zmiennych przed ich wyświetleniem. Dodaje się je znakiem pipelingu `|`.

FILTR	PRZYKŁAD	WYNIK
lower	{{ imie lower }}	Zmienia na małe litery: "jan"
upper	{{ imie upper }}	Zmienia na wielkie litery: "JAN"
length	{{ lista length }}	Zwraca rozmiar listy/stringa
default	{{ adres default:"Brak danych" }}	Gdy zmienna fałszywa (None/False), wyświetla alternatywę
date	{{ data date:"Y-m-d" }}	Formatuje datę, np. "2023-10-15"
truncatewords	{{ artykuł truncatewords:30 }}	Obcina tekst do 30 słów i dodaje "..."

4.6. Dziedziczenie szablonów

To najważniejsza koncepcja szablonów! Nie kopiujemy struktury HTML na każdej stronie. Tworzymy JEDEN główny szablon (`base.html`), a pozostałe dziedziczą po nim, nadpisując tylko wybrane bloki.

Krok 1: Główny szablon z zadeklarowanymi blokami (`templates/base.html`):

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block tytuł %}Tytuł domyślny{% endblock %}</title>
</head>
<body>
    <nav>Pasek nawigacji wspólny dla wszystkich</nav>

    <main>
        <!-- Tu wpadnie zawartość z innych plików -->
        {% block zawartosc %}{% endblock %}
    </main>

    <footer>Wspólna stopka</footer>
</body>
</html>
```

Krok 2: Szablon podstrony dziedziczący (`templates/o_nas.html`):

```
{% extends "base.html" %} <!-- To ZAWSZE musi być pierwsza linijka! -->

{% block tytuł %}O nas{% endblock %}

{% block zawartosc %}
    <h1>Witaj na stronie o nas!</h1>
    <p>To jest unikalna treść tylko dla tej podstrony.</p>
{% endblock %}
```

4.7. Dołączanie fragmentów

Czasami mamy fragment HTML powtarzający się wielokrotnie (np. karta pojedynczego artykułu na liście). Możemy go wyciągnąć do osobnego pliku i dołączać (inkludować).

```

<div class="lista-artykulow">
    {% for artykul in lista_artykulow %}
        <!-- Wstawia zawartość pliku przekazując mu bieżącą zmienną artykul -->
        {% include "fragmenty/karta_artykulu.html" with post=artykul %}
    {% endfor %}
</div>

```

4.8. Pliki statyczne (CSS, JS, obrazy)

Pliki statyczne to takie, które nie zmieniają się serwerowo, tylko ładują bezpośrednio w przeglądarce. By użyć ich w Django:

1. W pliku HTML załaduj moduł na samej górze: `{% load static %}`
2. Podaj ścieżkę generowaną tagiem `{% static '...' %}`

Przykład (`templates/base.html`):

```

{% load static %} <!-- Musi być na samej górze (zaraz po extends) -->
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
    
    <script src="{% static 'js/main.js' %}"></script>
</body>
</html>

```

4.9. Komentarze w szablonach

Zwykłe komentarze HTML `<!-- ... -->` wędrują do przeglądarki użytkownika. Komentarze Django są wycinane po stronie serwera!

```

{# To jest komentarz jednoliniowy – użytkownik go nie zobaczy #}

{% comment %}
To jest komentarz wieloliniowy.
Może zawierać nie działający kod!
{% for i in 10 %} ... {% endfor %}
{% endcomment %}

```

5. Modele (Models) — warstwa danych

5.1. Czym jest model

Model w Django to jedyne, rzetelne źródło prawdy o twoich danych. To zwykła klasa w Pythonie dziedzicząca po `models.Model`. Każdy model mapuje się na dokładnie jedną tabelę w bazie danych. Atrybuty klasy są kolumnami tabeli.

Zaletą ORM w Django jest to, że nie musisz znać języka SQL. Zmiana bazy z SQLite na PostgreSQL zajmuje kilka linijek kodu.

5.2. Definiowanie modelu

Modele tworzymy wewnątrz pliku `models.py` wybranej aplikacji.

blog/models.py :

```
from django.db import models

class Produkt(models.Model):
    nazwa = models.CharField(max_length=100)
    cena = models.DecimalField(max_digits=8, decimal_places=2)
    opis = models.TextField(blank=True)
    aktywny = models.BooleanField(default=True)
    utworzono = models.DateTimeField(auto_now_add=True)

    # Klasa Meta zawiera ustawienia dodatkowe tabeli
    class Meta:
        verbose_name = "Produkt"
        verbose_name_plural = "Produkty"
        ordering = ['-utworzono'] # Domyślne sortowanie od najnowszych

    # Metoda str to tekstowa reprezentacja obiektu (widoczna w Adminie!)
    def __str__(self):
        return f"{self.nazwa} ({self.cena} PLN)"
```

5.3. Typy pól

Django oferuje wiele gotowych typów kolumn.

POLE	TYP W BAZIE (NP. SQL)	OPIS
CharField	VARCHAR	Krótki tekst. Wymaga parametru <code>max_length</code> .
TextField	TEXT	Długi tekst.
IntegerField	INTEGER	Liczba całkowita.
DecimalField	DECIMAL	Liczba do obliczeń (wymaga <code>max_digits</code> , <code>decimal_places</code>). Idealne do pieniędzy.
FloatField	FLOAT	Zwykła liczba zmiennoprzecinkowa.
BooleanField	BOOLEAN	Prawda / Fałsz.
DateField	DATE	Sama data.
DateTimeField	DATETIME	Data z godziną. Można ustawić <code>auto_now_add=True</code> .
EmailField	VARCHAR	Waliduje, czy string jest poprawnym emailiem.
FileField / ImageField	VARCHAR	Przechowuje ścieżkę do pliku/zdjęcia.

5.4. Opcje pól

Każde pole posiada opcje, które definiują jak zachowuje się ono w bazie i formularzach.

- `null=True` : Pozwala na NULL w bazie.
- `blank=True` : Pozwala na puste pola formularzy.
- `default=X` : Wartość domyślna.
- `unique=True` : Nie można dodać dwóch takich samych wartości.
- `choices=TUPLE` : Tworzy listę wyboru.

Przykład choices:

```
STATUS_ZAMOWIENIA = (
    ('nowe', 'Nowe (Oczekujące)'),
    ('wyslane', 'Wysłane'),
    ('zrealizowane', 'Zrealizowane'),
)

status = models.CharField(max_length=20, choices=STATUS_ZAMOWIENIA, default='nowe')
```

5.5. Klucz główny (Primary Key)

Domyślnie Django tworzy każdemu modelowi klucz główny (ID), który automatycznie rośnie:

```
id = models.BigAutoField(primary_key=True)
```

5.6. Migracje

Samo zapisanie kodu w `models.py` nie zmienia bazy danych. Musisz stworzyć "migrację" i ją uruchomić!

KROK 1. Stworzenie plików migracji:

```
python manage.py makemigrations
```

KROK 2. Zastosowanie migracji w bazie danych:

```
python manage.py migrate
```

5.7. Relacje: Wiele do jednego (ForeignKey)

Najpopularniejszy typ relacji. Wiele artykułów należy do jednej kategorii.

`models.py`:

```
class Kategoria(models.Model):
    nazwa = models.CharField(max_length=50)

class Artykuł(models.Model):
    tytuł = models.CharField(max_length=100)
    # Wskazanie, że Artykuł należy do jednej Kategorii.
    kategoria = models.ForeignKey(Kategoria, on_delete=models.CASCADE)
```

5.8. Relacje: Wiele do wielu (ManyToManyField)

Artykuł może mieć wiele tagów. Tag może należeć do wielu artykułów.

```
class Tag(models.Model):
    nazwa = models.CharField(max_length=30)

class Wpis(models.Model):
    tytuł = models.CharField(max_length=100)
```



```
# Pole ManyToMany dodajemy tylko z jednej strony!  
tagi = models.ManyToManyField(Tag, blank=True)
```

5.9. Relacje: Jeden do jednego (OneToOneField)

Gdy rozszerzamy model, np. dodając Profil Użytkownika.

```
from django.contrib.auth.models import User  
  
class ProfilUzytkownika(models.Model):  
    user = models.OneToOneField(User, on_delete=models.CASCADE)  
    bio = models.TextField()
```

5.10. Opcja on_delete

Przy relacjach określamy, co ma się stać przy usuwaniu rodzica.

- `models.CASCADE`: Usuń też wszystkie powiązane dzieci! (domyślnie).
- `models.PROTECT`: Zablokuj usuwanie, dopóki dzieci istnieją.
- `models.SET_NULL`: Ustaw pole dziecka na NULL (`null=True` wymagane).
- `models.SET_DEFAULT`: Ustaw na domyślną wartość.
- `models.DO_NOTHING`: Nie rób nic.

5.11. Praca z istniejącą bazą danych (inspectdb)

Django oferuje doskonałe narzędzie do inżynierii odwrotnej, gdy mamy już gotową, utworzoną wcześniej bazę danych (z istniejącymi tabelami) i chcemy zintegrować ją z naszym projektem, by korzystać z niej poprzez ORM.

Zamiast pisać modele ręcznie, Django potrafi automatycznie przeanalizować podłączoną bazę i wygenerować odpowiadający jej kod dla pliku `models.py`.

Krok 1: Podpięcie bazy w `settings.py` Najpierw upewnij się, że w pliku konfiguracyjnym w słowniku `DATABASES` znajdują się prawidłowe parametry połączenia z docelową bazą (np. ścieżka do pliku SQLite, lub poświadczenia dla MySQL/PostgreSQL).

Krok 2: Użycie komendy `inspectdb` Aby zobaczyć wygenerowane modele w konsoli (na próbę), wpisz polecenie:

```
python manage.py inspectdb
```

Aby od razu zapisać ten wynik bezpośrednio w pliku `models.py` Twojej aplikacji, przekieruj strumień wyjścia (pamiętaj, że nadpisze to obecny plik):

```
python manage.py inspectdb > nazwa_aplikacji/models.py
```

Krok 3: Analiza wygenerowanego pliku Po utworzeniu pliku warto do niego zajrzeć. Zauważysz tam m.in.:

- Atrybut `managed = False` w wewnętrznej klasie `Meta` każdego modelu. Mówi to Django, by zachowało tę tabelę tylko do odczytu i nie próbowało wykonywać na niej migracji (tworzyć/usuwać tabeli czy jej kolumn). Jeśli chcesz w pełni zarządzać cyklem życia tej tabeli z poziomu Django, powinieneś zmienić to na `managed = True` (i wykonać `makemigrations`).
- Czasami wygenerowane modele mogą wymagać drobnych poprawek ręcznych, zwłaszcza jeśli konwencje nazewnictwa w bazie były bardzo nietypowe.

Dzięki `inspectdb` podłączenie istniejących tabel do nowego projektu Django staje się niezwykle proste.

6. ORM — zapytania do bazy danych

6.1. Czym jest ORM

ORM (Object-Relational Mapping) to technika programowania, która pozwala przetwarzać dane w relacyjnej bazie danych tak, jakby były obiektami w kodzie Pythona.

Django ORM pozwala tworzyć, pobierać, aktualizować i usuwać rekordy (tzw. operacje CRUD) bez pisania ani jednej linijki surowego kodu SQL. Zapewnia to również automatyczną ochronę przed atakami SQL Injection.

6.2. Tworzenie obiektów

Są dwa główne sposoby na dodanie nowego rekordu do bazy.

Sposób 1: Utworzenie obiektu i wywołanie `.save()`

```
# 1. Tworzy instancję obiektu w pamięci RAM Pythona
nowy_produkt = Produkt(nazwa="Klawiatura", cena=150.00)

# 2. Fizycznie zapisuje (wykonuje INSERT) do bazy danych
nowy_produkt.save()
```

Sposób 2: Wywołanie `objects.create()` (Krótsze, lepsze)

```
# Od razu tworzy i zapisuje do bazy danych
Produkt.objects.create(nazwa="Myszka", cena=80.00)
```

6.3. Pobieranie wszystkich rekordów

```
# Zwraca QuerySet (listę obiektów) ze wszystkimi rekordami z tabeli
wszystkie_produkty = Produkt.objects.all()

for p in wszystkie_produkty:
    print(p.nazwa, p.cena)
```

6.4. Pobieranie jednego rekordu

Gdy chcemy pobrać tylko jeden konkretny obiekt:

Metoda `get()` Pobiera dokładnie jeden rekord. Używaj jej, gdy szukasz po unikalnym polu (np. `id`).

```
try:
    produkt = Produkt.objects.get(id=5)
    print(produkt.nazwa)
except Produkt.DoesNotExist:
    print("Nie znaleziono takiego produktu!")
except Produkt.MultipleObjectsReturned:
    print("Znaleziono więcej niż jeden! (przy get() to błąd)")
```

Metody `first()` i `last()` Bezpieczniejsze alternatywy dla `.get()`. Zwracają pierwszy (lub ostatni) element pasujący do zapytania, lub `None`, jeśli nic nie znaleziono (nie rzucają błędami!).

```
# Pobiera pierwszy produkt (domyślnie według kolejności z class Meta: ordering)
pierwszy = Produkt.objects.first()
```

```
if pierwszy:
    print(pierwszy.nazwa)
```

6.5. Filtrowanie

Służą do tego metody `filter()` oraz `exclude()`.

```
# filter: Zwraca tylko te rekordy, które spełniają warunek (aktywny = True)
aktywne = Produkt.objects.filter(aktywny=True)

# Można łączyć warunki po przecinku (działają jak logiczne AND)
tanie_aktywne = Produkt.objects.filter(aktywny=True, cena=50.00)

# exclude: Zwraca wszystkie OPRÓCZ tych, które spełniają warunek
nieaktywne = Produkt.objects.exclude(aktywny=True)
```

6.6. Lookup'y — zaawansowane filtrowanie

W surowym SQL używamy operatorów typu `>`, `<`, `LIKE`, `IN`. W Django ORM stosujemy "lookups", oddzielając nazwę pola podwójnym podkreśleniem `__`.

LOOKUP	OPIS	PRZYKŁAD W DJANGO	ODPOWIEDNIK SQL
<code>__exact</code>	Dokładne dopasowanie	<code>filter(nazwa__exact="Mysz")</code>	<code>WHERE nazwa = 'Mysz'</code>
<code>__contains</code>	Zawiera fragment	<code>filter(nazwa__contains="Pro")</code>	<code>WHERE nazwa LIKE '%Pro%'</code>
<code>__icontains</code>	Jak wyżej, ale ignoruje wielkość liter	<code>filter(nazwa__icontains="pro")</code>	<code>ILIKE '%pro%'</code>
<code>__startswith</code>	Zaczyna się od	<code>filter(nazwa__startswith="Mac")</code>	<code>LIKE 'Mac%'</code>
<code>__gt</code>	Większe niż (Greater Than)	<code>filter(cena__gt=100)</code>	<code>WHERE cena > 100</code>
<code>__gte</code>	Większe lub równe	<code>filter(cena__gte=100)</code>	<code>WHERE cena ≥ 100</code>
<code>__lt</code>	Mniejsze niż (Less Than)	<code>filter(cena__lt=50)</code>	<code>WHERE cena < 50</code>
<code>__in</code>	Należy do listy	<code>filter(id__in=[1, 2, 3])</code>	<code>WHERE id IN (1,2,3)</code>
<code>__range</code>	W zakresie	<code>filter(cena__range=(50, 100))</code>	<code>BETWEEN 50 AND 100</code>
<code>__isnull</code>	Czy jest NULL	<code>filter(opis__isnull=True)</code>	<code>WHERE opis IS NULL</code>

6.7. Sortowanie

Do sortowania wyników używamy `order_by()`. Dodanie znaku `-` przed nazwą pola odwraca sortowanie (malejąco).

```
# Od najtańszego do najdroższego
rosnaco = Produkt.objects.all().order_by('cena')

# Od najdroższego
malejaco = Produkt.objects.all().order_by('-cena')

# Sortowanie po kilku kolumnach (najpierw po nazwie, w przypadku remisów po cenie malejąco)
zlozone = Produkt.objects.all().order_by('nazwa', '-cena')
```

6.8. Ograniczanie wyników

W Django nie ma metody `limit()` ani `offset()`. Używamy zwykłego list slicing'u Pythona:

```
# Pobierz tylko pierwsze 5 wyników (SQL: LIMIT 5)
top_5 = Produkt.objects.all().order_by('-cena')[:5]

# Pobierz wyniki od 5 do 15 (SQL: LIMIT 10 OFFSET 5)
kolejne = Produkt.objects.all()[5:15]

# Aby policzyć ilość, używaj count() zamiast len()!
ile = Produkt.objects.filter(aktywny=True).count() # Wydażne! SQL: SELECT COUNT(*)

# By sprawdzić, czy jakiegolwiek rekord istnieje, użyj exists()
czy_sa = Produkt.objects.filter(cena__lt=10).exists() # Zwraca True/False
```

6.9. Aktualizacja rekordów

Aktualizacja jednego obiektu:

```
p = Produkt.objects.get(id=1)
p.cena = 200.00
p.save() # Zapisuje zmianę w bazie
```

Masowa aktualizacja (dużo szybsza):

```
# Zmienia cenę wszystkich nieaktywnych produktów na 0
Produkt.objects.filter(aktywny=False).update(cena=0)
```

6.10. Usuwanie rekordów

```
# Usunięcie pojedynczego obiektu
p = Produkt.objects.get(id=1)
p.delete()

# Masowe usuwanie (uwaga: jest natychmiastowe!)
Produkt.objects.filter(aktywny=False).delete()

# Usunięcie wszystkiego w tabeli (niebezpieczne!)
Produkt.objects.all().delete()
```

6.11. Łączenie zapytań (chaining)

Zapytania ORM można łączyć w długie łańcuchy (tzw. method chaining). Zapytanie SQL jest generowane (i wysyłane do bazy) dopiero w momencie, gdy faktycznie iterujemy po wynikach. Zjawisko to nazywa się **Lazy Evaluation** (leniwe ewaluowanie).

```
wynik = Produkt.objects.filter(aktywny=True)\
    .exclude(nazwa__contains="Test")\
    .order_by('-cena')[:3]
```

6.12. Zapytania na relacjach

Podwójne podkreślenie `__` pozwala również na "przechodzenie" przez relacje bez pisania skomplikowanych JOIN-ów.

Mamy modele `Kategoria` oraz `Artykuł(kategoria=ForeignKey(Kategoria))`.

Z poziomu Dziecka do Rodzica:

```
# Zwróć wszystkie artykuły, których kategoria ma nazwę "Technologia"
artykuły = Artykuł.objects.filter(kategoria__nazwa="Technologia")
```

Z poziomu Rodzica do Dzieci (related_name): Jeśli w modelu `Artykuł` zdefiniowaliśmy relację: `kategoria = models.ForeignKey(Kategoria, related_name='artykuły')` To obiekt kategorii zyskuje ukryte pole dostępu:

```
kat = Kategoria.objects.get(nazwa="Muzyka")
# Pobranie wszystkich artykułów należących do tej kategorii
wszystkie_jej_posty = kat.artykuły.all()
```

7. Panel administracyjny (Django Admin)

7.1. Tworzenie superużytkownika

Panel admina to gotowy system CMS do zarządzania Twoją aplikacją. By uzyskać do niego dostęp, musisz posiadać konto.

```
python manage.py createsuperuser
```

(Zostaniesz poproszony o nazwę, email i dwukrotnie o hasło - w systemach Unix wpisywane hasło jest niewidoczne!)

Po utworzeniu uruchom serwer i wejdź na `http://127.0.0.1:8000/admin/`.

7.2. Rejestrowanie modeli w admin.py

Aby Twój model (np. `Produkt`) pojawił się w panelu, musisz go zarejestrować.

`blog/admin.py`:

```
from django.contrib import admin
from .models import Produkt

admin.site.register(Produkt)
```

Od teraz możesz dodawać, edytować i usuwać produkty z poziomu wygodnego interfejsu graficznego.

7.3. Klasa ModelAdmin

Podstawowa rejestracja daje tylko listę obiektów opartą na wyniku funkcji `__str__()`. Możemy dostosować wygląd za pomocą klasy dziedziczącej po `admin.ModelAdmin`.

`blog/admin.py`:

```
from django.contrib import admin
from .models import Produkt
```

```
class ProduktAdmin(admin.ModelAdmin):
    # Kolumny widoczne na liście wszystkich elementów
    list_display = ('nazwa', 'cena', 'aktywny', 'utworzono')

    # Dodaje panel z boku do filtrowania po podanych kolumnach
    list_filter = ('aktywny', 'utworzono')

    # Dodaje pasek wyszukiwania (szuka we wskazanych polach)
    search_fields = ('nazwa', 'opis')

    # Domyślne sortowanie (tu: najpierw aktywne, potem najdroższe)
    ordering = ('-aktywny', '-cena')

    # Zmieniona rejestracja! Podajemy model i naszą klasę ustawień
    admin.site.register(Produkt, ProduktAdmin)
```

7.4. Edycja widoku szczegółów

Możemy też konfigurować to, co widzimy *wewnątrz* pojedynczego rekordu.

```
class ProduktAdmin(admin.ModelAdmin):
    list_display = ('nazwa', 'cena', 'aktywny')

    # Pola, których administrator NIE MOŻE edytować
    readonly_fields = ('utworzono',)

    # Grupowanie pól w ładne sekcje z nagłówkami
    fieldsets = (
        ('Podstawowe informacje', {
            'fields': ('nazwa', 'opis')
        }),
        ('Opcje i ceny', {
            'fields': ('cena', 'aktywny')
        }),
        ('Metadane', {
            'classes': ('collapse',), # Sekcja domyślnie zwinięta
            'fields': ('utworzono',)
        }),
    )
```

8. Formularze (Forms)

8.1. Czym jest formularz

Pobieranie danych bezpośrednio z `request.POST` jest uciążliwe i bardzo niebezpieczne (brak walidacji). Django oferuje system Formularzy, który wykonuje 3 zadania:

1. Automatycznie generuje bezpieczny kod HTML (tagi `<input>`, `<select>`).
2. Przeprowadza walidację i oczyszcza dane.
3. Wyświetla zlokalizowane komunikaty o błędach obok niepoprawnych pól.

8.2. Definiowanie formularza

Formularze zazwyczaj tworzymy w nowym pliku `forms.py` obok `models.py`.

`blog/forms.py`:

```
from django import forms

class KontaktForm(forms.Form):
    # Pola deklarujemy podobnie jak w modelach!
    imie = forms.CharField(max_length=50, label="Twoje imię")
    email = forms.EmailField(label="Adres e-mail")
    temat = forms.CharField(max_length=100)
    wiadomosc = forms.CharField(widget=forms.Textarea, label="Treść")
    zgoda_rodz = forms.BooleanField(required=True, label="Akceptuję regulamin")
```

8.3. Obsługa formularza w widoku (FBV)

Aby obsłużyć formularz, musimy rozróżnić zapytanie typu GET (ktoś wszedł na stronę i chce zobaczyć pusty formularz) od POST (ktoś kliknął "Wyślij").

`blog/views.py`:

```
from django.shortcuts import render, redirect
from django.contrib import messages
from .forms import KontaktForm

def strona_kontaktowa(request):
    if request.method == 'POST':
        # Instancjujemy formularz z danymi, które podesłał użytkownik
        form = KontaktForm(request.POST)

        # is_valid() sprawdza wszystkie reguły (max_length, poprawność email, itp.)
        if form.is_valid():
            # Po walidacji, wyczyszczone, poprawne dane lądują w cleaned_data
            imie = form.cleaned_data['imie']
            email = form.cleaned_data['email']
            wiadomosc = form.cleaned_data['wiadomosc']

            # W tym miejscu np. wysyłamy email
            print(f"Wiadomość od {imie} ({email}): {wiadomosc}")

            # Przekierowujemy po sukcesie na tę samą stronę, zapobiega podwójnemu POST
            return redirect('start')

        else:
            # Puste żądanie GET, tworzymy pusty formularz
            form = KontaktForm()

    # Niezależnie czy GET (pusty formularz) czy POST z błędami, renderujemy go:
    return render(request, 'kontakt.html', {'form': form})
```

8.4. Wyświetlanie formularza w szablonie

Podczas budowania formularza w HTML, musimy ręcznie napisać tag `<form>` oraz guzik "Wyślij". Sam Django generuje tylko środkowe `<input>`.

Obowiązkowy jest tag zabezpieczający `{% csrf_token %}` !

`templates/kontakt.html` :

```
<form method="POST" action="">
    {% csrf_token %}

    <!-- Renderuje pola formularza jako paragrafy <p> -->
    {{ form.as_p }}

    <!-- Inne metody:
    {{ form.as_table }}
    {{ form.as_div }} (w najnowszych wersjach Django)
    -->

    <button type="submit">Wyślij wiadomość</button>
</form>
```

8.5. Ręczne renderowanie formularza

Gdy chcemy mieć pełną kontrolę nad Bootstrapem lub własnym CSS, iterujemy po polach ręcznie:

```
<form method="POST">
    {% csrf_token %}

    {% for pole in form %}
        <div class="mb-3">
            {{ pole.label_tag }} <!-- Renderuje etykietę -->
            {{ pole }} <!-- Renderuje samego inputa -->

            {% if pole.errors %} <!-- Obsługa błędów np. na czerwono -->
                <div style="color: red;">
                    {% for error in pole.errors %}
                        <small>{{ error }}</small>
                    {% endfor %}
                </div>
            {% endif %}

            {% if pole.help_text %}
                <small style="color: gray;">{{ pole.help_text }}</small>
            {% endif %}
        </div>
    {% endfor %}

    <button type="submit">Zapisz</button>
</form>
```


8.6. ModelForm — szybki formularz na bazie modelu

Bardzo często tworzymy formularz tylko po to, by dodać nowy rekord do tabeli (np. tworzenie Artykułu). Pisanie klas formularza od zera mija się z celem, bo dublujemy pracę z `models.py`. Wykorzystujemy do tego klasę `ModelForm`.

`blog/forms.py`:

```
from django import forms
from .models import Artykuł

class ArtykułForm(forms.ModelForm):
    class Meta:
        model = Artykuł
        # Wskazujemy które pola z modelu mają stać się formularzem:
        fields = ['tytuł', 'tresc', 'kategoria']
        # LUB: fields = '__all__' (wybiera wszystkie)
        # LUB: exclude = ['utworzono'] (wszystkie oprócz utworzono)
```

Zapisywanie w widoku w przypadku ModelForm:

```
form = ArtykułForm(request.POST)
if form.is_valid():
    nowy_artykuł = form.save() # Automatycznie tworzy i zapisuje obiekt bazy!
```

8.7. Formularz z przesyłaniem plików

Gdy formularz zawiera pole wgrywania obrazów (`ImageField`) lub plików (`FileField`), musisz spełnić dwa absolutne warunki!

1. Szablon `<form>` musi mieć ustawiony `enctype`.
2. Do konstruktora formularza **musisz** podać w widoku `request.FILES`.

HTML:

```
←!— Jeśli nie dodasz multipart/form-data, pliki nigdy nie polecą do serwera! →
<form method="POST" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Wgraj</button>
</form>
```

Zmienna `views.py`:

```
def wgraj_plik(request):
    if request.method == 'POST':
        # request.FILES jest KONIECZNE dla przesyłania plików
        form = UploadForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            return redirect('sukces')
    else:
        form = UploadForm()
    return render(request, 'upload.html', {'form': form})
```

9. Widoki oparte na klasach (Class-Based Views)

9.1. Dlaczego CBV? FBV vs CBV

Do tej pory używaliśmy widoków opartych na funkcjach (FBV). Django pozwala też na używanie klas (CBV). Obie formy są poprawne, lecz mają inne zalety.

RODZAJ WIDOKU	ZALETY	KIEDY UŻYWAĆ?
FBV (Funkcje)	Proste w czytaniu. Jasny przepływ logiki (GET vs POST w jednym miejscu). Łatwe dla niestandardowych logik i endpointów.	Przy skomplikowanej logice (np. zapisywanie wielu rzeczy na raz), dla API, oraz gdy się uczysz Django.
CBV (Klasy)	Niesamowicie DRY (Don't Repeat Yourself). Gotowe "klocki" oszczędzają masę pisaniny. Kod jest zwięzły i łatwy w re-używaniu (dziedziczenie).	Do typowych i powtarzalnych operacji CRUD: listy obiektów, formularze, widok jednego rekordu.

Aby użyć CBV w pliku `urls.py`, zawsze na końcu nazwy klasy musimy wywołać `.as_view()`.

```
path('posty/', ArtykulyListView.as_view(), name='lista_postow'),
```

9.2. TemplateView — wyświetlanie szablonu

Najprostsza klasa służąca jedynie do wyrenderowania strony statycznej.

`views.py`:

```
from django.views.generic import TemplateView

class ONasView(TemplateView):
    # Wskazujemy tylko szablon - to wszystko, czego potrzebuje ta klasa!
    template_name = "strony/o_nas.html"
```

9.3. ListView — lista obiektów

Bardzo przydatny widok do wyświetlania listy wszystkich rekordów danego modelu.

`views.py`:

```
from django.views.generic import ListView
from .models import Artykuł

class ListaArtykułowView(ListView):
    model = Artykuł
    template_name = 'artykuly_lista.html' # domyślnie to nazwamodelu_list.html
    context_object_name = 'posty' # nazwa pod jaką lista trafi do HTML (domyślnie object_list)
    paginate_by = 10 # Automatyczna paginacja!

    # Można nadpisać get_queryset aby dodać np. filtrowanie
    def get_queryset(self):
        return Artykuł.objects.filter(opublikowany=True).order_by('-data')
```

9.4. DetailView — szczegóły jednego obiektu

Służy do pokazania jednego konkretnego obiektu na podstawie jego klucza podstawowego (`pk`) w adresie URL.

`urls.py`:

```
# Musi być <int:pk>, żeby DetailView wiedział czego szukać
path('artykul/<int:pk>/', SzczegolyArtykuLuView.as_view(), name='szczegoly'),
```

`views.py`:

```
from django.views.generic import DetailView
from .models import Artykul

class SzczegolyArtykuLuView(DetailView):
    model = Artykul
    template_name = 'artykul_szczegoly.html'
    context_object_name = 'artykul' # W szablonie wyświetlamy: {{ artykul.tytul }}
```

9.5. CreateView — tworzenie nowego obiektu

Widok generujący i obsługujący formularz tworzenia nowego obiektu (nie potrzebujemy nawet tworzyć pliku `forms.py` - CreateView wygeneruje ModelForm samo!).

`views.py`:

```
from django.views.generic import CreateView
from .models import Artykul
from django.urls import reverse_lazy

class DodajArtykulView(CreateView):
    model = Artykul
    fields = ['tytul', 'tresc', 'kategoria'] # Pola jakie mają być w formularzu
    template_name = 'formularz_artykuLu.html'

    # reverse_lazy przelicza url na adres WWW PO wczytaniu ustawień
    success_url = reverse_lazy('lista_postow') # Dokąd po dodaniu?
```

9.6. UpdateView — edycja istniejącego obiektu

Identyczny jak CreateView, ale wymaga `pk` w adresie URL, aby wiedzieć, który obiekt edytujemy. Pobiera stare dane, wypełnia nimi inputy, a po zatwierdzeniu nadpisuje.

`views.py`:

```
from django.views.generic import UpdateView
from .models import Artykul

class EdytujArtykulView(UpdateView):
    model = Artykul
    fields = ['tytul', 'tresc', 'kategoria']
    template_name = 'formularz_artykuLu.html'

    # Jeśli nie podasz success_url, Django spróbuje poszukać na modelu Artykul
    # metody get_absolute_url()
```

9.7. DeleteView — usuwanie obiektu

Generuje stronę z przyciskiem potwierdzającym usunięcie obiektu (jest to wymóg bezpieczeństwa - usunięcie rekordu powinno zawsze działać tylko z żądania POST).

`views.py`:

```
from django.views.generic import DeleteView
from .models import Artykuł
from django.urls import reverse_lazy

class UsunArtykułView(DeleteView):
    model = Artykuł
    template_name = 'potwierdz_usuniecie.html'
    success_url = reverse_lazy('lista_postow')
```

9.8. FormView — obsługa zwykłego formularza

Służy do obsługi formularzy, które niekoniecznie wywodzą się wprost z modelu bazy danych (czyli dla tych tworzonych przez zwykłe `forms.Form`). Idealny np. do obsługi wysłania maila kontaktowego.

```
from django.views.generic import FormView
from .forms import KontaktForm

class KontaktView(FormView):
    template_name = 'kontakt.html'
    form_class = KontaktForm # Nie podajemy 'model', podajemy pełną klasę formularza
    success_url = '/dziekujemy/'

    # Metoda wykonana jeśli dane są w 100% poprawne
    def form_valid(self, form):
        # Wysyłanie emaila
        email = form.cleaned_data['email']

        # return form_valid powoduje przekierowanie do success_url
        return super().form_valid(form)
```

9.9. Podsumowanie — magii CBV

Gdy spojrzymy na metody z klas opartych na widokach:

- `ListView` to `Model.objects.all()` ukryty pod maską, plus przekazanie `context['object_list']` do HTML.
- `DetailView` to pobranie zmiennej `pk` ze ścieżki i wykonanie `get_object_or_404(Model, pk=pk)`, a potem przekazanie obiektu w zmiennej do contextu.
- `CreateView` / `UpdateView` / `FormView` to całkowite zastąpienie bloku kodu z zagnieżdżonymi sprawdzaniami: `if request.method == 'POST', form = Form(request.POST) i if form.is_valid(): form.save()`.

Cały ten skomplikowany, ręczny kod, który powtarzalibyśmy przy każdym widoku bazodanowym, został spakowany w eleganckie, gotowe klasy konfiguracyjne. To potęga Django!

10. Uwierzytelnianie i autoryzacja

10.1. Wbudowany system auth

Zamiast samemu bawić się w hashowanie haseł i zarządzanie sesją po ciasteczkach, Django dostarcza w pełni funkcjonalny, bezpieczny, natywny system autentykacji. Aplikacja instaluje go automatycznie w pakiecie `django.contrib.auth`.

Centralnym punktem systemu jest domyślny Model `User`. Zawiera on z góry predefiniowane pola na: login (nazwę użytkownika), weryfikowalne hasło, e-mail, datę rejestracji, aktywność oraz to, czy użytkownik to admin.

10.2. Wbudowane formularze logowania i wylogowanie

Żeby zbudować sprawnie działający proces logowania i wylogowania, nie musimy w zasadzie pisać widoków - wbudowane widoki CBV robią to idealnie. Musimy dodać tylko kilka importów w naszym globalnym pliku URL-i.

`mojprojekt/urls.py`:

```
from django.contrib.auth import views as auth_views

urlpatterns = [
    # Wbudowany widok do logowania - domyślnie szuka na ścieżce: registration/login.html
    path('logowanie/', auth_views.LoginView.as_view(template_name='logowanie.html'), name='login'),

    # Wbudowany widok wylogowywania (w najnowszych wersjach wywołuje się go POSTem!)
    path('wyloguj/', auth_views.LogoutView.as_view(), name='logout'),
]
```

Oczywiście do logowania musimy przygotować sam plik `logowanie.html` ze standardowym generatorem `{{ form.as_p }}` oraz określić przekierowanie po udanym logowaniu (w pliku `settings.py`).

`settings.py`:

```
# Po zalogowaniu przerzuca na ścieżkę o nazwie 'start'
LOGIN_REDIRECT_URL = 'start'

# Po wylogowaniu przerzuca na ścieżkę o nazwie 'login'
LOGOUT_REDIRECT_URL = 'login'
```

10.3. Rejestracja użytkownika — UserCreationForm

Dla stworzenia rejestracji z wbudowanym walidowaniem dwukrotnie wprowadzanego poprawnego hasła - korzystamy ze wspomnianego wcześniej wbudowanego formularza `UserCreationForm`.

Możemy użyć do tego widoku typu `CreateView`! Poniżej kompletny plik:

`users/views.py`:

```
from django.views.generic import CreateView
from django.contrib.auth.forms import UserCreationForm
from django.urls import reverse_lazy

class RejestracjaView(CreateView):
    # Do tego formularza NIE przypinamy modelu (jest już w nim zaszyty Model User).
    form_class = UserCreationForm
    template_name = 'rejestracja.html'
    success_url = reverse_lazy('login')
```

10.4. Ograniczanie dostępu do stron dla niezalogowanych

To niezwykle kluczowe! Nikt niepowołany nie powinien modyfikować systemu. Poniżej omówimy metody blokujące.

Dla widoków typu FBV (Funkcji): - dekorator `@login_required`

```
from django.contrib.auth.decorators import login_required

# Jeśli user wejdzie tu bez bycia zapamiętanym sesją, zostanie z automatu
# wyrzucony na stronę /logowanie/ (zdefiniowaną w urls i settings).
@login_required
def panel_uzytkownika(request):
    return render(request, 'tajny_panel.html')
```

Dla widoków typu CBV (Klas): - mixin `LoginRequiredMixin`

W Pythonie istnieje możliwość "dziedziczenia wielokrotnego". W widokach klas dodajemy na PIERWSZYM MIEJSCU w dziedziczeniu klasę-mixin, która "wmiksuje" do widoku sprawdzanie uprawnień.

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView

class TajnaListaOczenView(LoginRequiredMixin, ListView):
    model = Ocena
    template_name = 'oceny.html'
```

10.5. Grupy i uprawnienia dla pracowników

Oprócz zwykłego flagowania `is_authenticated`, Django wdraża pojęcie uprawnień (`Permissions`). Panel administracyjny domyślnie generuje po 4 permisie dla każdego stworzonego Modelu (add, change, delete, view).

Aby użyć w kodzie uprawnienia do wejścia w widok FBV, użyj dekoratora sprawdzania:

```
from django.contrib.auth.decorators import permission_required

# Przepuści używającego ten widok, tylko gdy ma pozwolenie edycji w aplikacji 'blog' dla modelu 'artykuł'.
@permission_required('blog.change_artykuł')
def modyfikuj_artykuly(request):
    pass
```

Dla widoku typu klasa użyj `PermissionRequiredMixin`:

```
from django.contrib.auth.mixins import PermissionRequiredMixin

class SkasujUzytkownika(PermissionRequiredMixin, DeleteView):
    permission_required = 'auth.delete_user'
```

10.6. Dostęp do użytkownika w szablonie

Ponieważ Django implementuje "procesory kontekstu", w KAŻDYM wyrenderowanym w systemie pliku HTML masz natychmiastowy dostęp do obiektu bieżącego gościa za pomocą globalnej zmiennej `{{ user }}` (nawet nie przekazując jej w widoku w `render(dane)`!).

```

<!-- Czy użytkownik ma sesję zalogowanego w systemie (niezależnie od adminów) -->
{% if user.is_authenticated %}

    <p>Zalogowano pomyślnie. Witaj, {{ user.username }}!</p>

    <a href="{% url 'logout' %}">Wyloguj się</a>

{% else %}

    <p>Oglądasz tę stronę jako gość.</p>

    <a href="{% url 'login' %}">Zaloguj się!</a>

{% endif %}

```

Możemy również warunkować konkretne uprawnienia do elementów HTMLowych i bloków (np. ukrywać guzik Dodaj artykuł przed resztą, by nikogo nie frustrował "odmawianiem" w widoku).

```

{% if perms.blog.add_artikul %}

    <button>Dodaj nowy artykuł</button>

{% endif %}

```

10.7. Rozszerzanie modelu User — OneToOne

Bardzo często użytkownicy potrzebują posiadać np. opis profilowy, rok urodzenia, avatar, lub numer identyfikacyjny z systemu z zewnątrz. Tworzenie całego nowego modelu autentykacji (wymaga głębokiego wprowadzania tzw. `AbstractUser`) to dla wielu spory nadmiar kłopotów. Świetnym i powszechnym sposobem na powiększenie danych jest stworzenie tzw. Modelu Rozszerzającego "OneToOne".

```

from django.db import models
from django.contrib.auth.models import User

# Własny model
class Profil(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    avatar = models.ImageField(upload_to='avatars/', default='default.jpg')
    opis = models.TextField()

    def __str__(self):
        return f'Profil konta: {self.user.username}'

```

Dzięki potędze ORM'u, wyciąganie obrazka jest proste: W pliku html: `{{ user.profil.avatar.url }}` - nie trzeba tworzyć nowych zapytań!

11. Pliki statyczne i media

11.1. Różnica między static a media

W ekosystemie Django pliki serwowane użytkownikowi dzielimy na dwie kategorie:

- Pliki statyczne (Static)** - pliki nierozdzielnie związane z kodem aplikacji (pliki CSS, JavaScript, logo strony, ikony). Są tworzone przez developera i zarządzane w systemie kontroli wersji (Git).
- Pliki multimedialne / Media (Media/Uploads)** - pliki wgrywane (uploadowane) w sposób dynamiczny przez użytkowników strony lub przez panel administratora (zdjęcia profilowe, załączniki, wgrywane obrazy artykułów). Te pliki NIE wchodzą do systemu kontroli wersji!

11.2. Konfiguracja plików statycznych

Zwykle konfiguracja domyślna pozwala na lokalne ładowanie zasobów w każdym folderze `/static/` należącym do aplikacji. Najczęściej jednak w małych projektach deweloperzy chcą posiadać jeden zbiorczy folder `static` w głównym katalogu (tam gdzie `manage.py`).

`settings.py`:

```
import os

# Pod jakim URL-em w przeglądarce będą widoczne (np. http://localhost:8000/static/css/style.css)
STATIC_URL = '/static/'

# Gdzie Django ma szukać plików współdzielonych (dodatkowych katalogów statycznych)
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'static'),
]

# Katalog, do którego wylądują WSZYSTKIE pliki (zbierane z paczek i aplikacji) po komendzie 'collectstatic' - używane wyłącznie
# podczas wdrażania (Deploymentu)
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
```

11.3. Używanie CSS i JS w szablonie

```
←!— Obowiązkowo w pierwszej linijce pliku (po extends) wgrywamy silnik tagu →
{% load static %}

<!DOCTYPE html>
<html>
<head>
    ←!— Tag wygeneruje adres: /static/css/style.css →
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
    <script src="{% static 'js/main.js' %}"></script>
</body>
</html>
```

11.4. Konfiguracja plików Media (Wgrywanych)

Wgrywanie plików różni się od plików statycznych. System musi wiedzieć na jakim dysku fizycznym odkładać pliki wrzucane za pomocą pól `ImageField` oraz `FileField`.

`settings.py`:

```
# Pod jakim URL-em przeglądarka dostanie się do załącznika
MEDIA_URL = '/media/'

# Fizyczny folder dyskowy na serwerze, do którego trafią uploady
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```


11.5. Serwowanie plików Media na serwerze deweloperskim

Pliki dodawane przez użytkownika nie są z automatu widoczne, jak widoki czy szablony! Django ze względów bezpieczeństwa nie serwuje plików Media w środowisku produkcyjnym, ale w ramach deweloperki lokalnej, musimy dopiąć małą magiczną "wstawkę" do głównego pliku `urls.py` projektu, by widzieć wgrywane obrazki!

`mojprojekt/urls.py` :

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('blog.urls')),
]

# Magiczna formuła - DODAJEMY TRASY dla mediów - ODPALANE TYLKO gdy DEBUG=True!
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Dzięki temu wpisowi wejście na wygenerowany w bazie URL np. `/media/avatars/ja.png` spowoduje wczytanie fizycznego pliku z dysku.

12. Środowisko uruchomieniowe i bezpieczeństwo

12.1. Zmienna DEBUG i ukrywanie błędów

Pamiętaj, że Django dzieli pracę nad kodem na dwa stany, całkowicie definiowane w pliku `settings.py`.

```
# DEBUG = True wyświetla tzw. "żółte ekrany śmierci" - przepotężne
# wiersze z kodem źródłowym błędu i wszystkimi zmiennymi w pamięci RAM.
DEBUG = True
```

ABSOLUTNIE ZABRONIONE JEST POZOSTAWIENIE DEBUG = TRUE NA PRODUKCJI. Jeżeli upubliczniasz projekt, powinieneś zmienić to na `DEBUG = False`. Od tego momentu po wywołaniu błędu przez system pokaże się jedynie bardzo krótki napis w rodzaju "Server Error 500" i ani grama kodu. Chroni to Twoją bazę danych przez wglądem.

Gdy `DEBUG = False`, obowiązkowo musisz wypełnić ustawienie `ALLOWED_HOSTS`:

```
# Dopuszczalne domeny Twojego systemu (zapobiega atakom HTTP Host Header).
ALLOWED_HOSTS = ['moj-sklep.pl', 'www.moj-sklep.pl', '127.0.0.1']
```

12.2. Secret Key (Klucz kryptograficzny)

Plik `settings.py` w momencie stworzenia projektu automatycznie zyskał niezwykle długi klucz `SECRET_KEY`.

```
SECRET_KEY = 'django-insecure-$9$8#_to86^k#^$u(s3p...'
```

Klucz ten używany jest by podpisywać "Ciasteczka", zarządzać systemem resetu haseł oraz ochroną w procesach szyfrujących frameworku. Jeśli ktoś przechwyci z GitHuba Twój `SECRET_KEY`, de facto może sfałszować sesję każdego administratora witryny! Zawsze trzymaj go w Zmiennych Środowiskowych! (Na przykład wykorzystując popularną bibliotekę `python-dotenv`).

12.3. Ochrona CSRF (Cross Site Request Forgery)

Django z zasady zabezpiecza wszystkie operacje edytujące bazę (żądania typu POST, PUT, DELETE). Domyślnie używa do tego włączonego pośrednika walidacyjnego `CsrfViewMiddleware`.

Bez zamieszczonego specjalnego ukrytego inputu - tokena `{% csrf_token %}` we wnętrzu HTMLa `<form>`, Django wygeneruje potężny **błąd 403 Forbidden - Odrzucono operację**. Atakujący nie mógłby podrobić żądania logowania lub modyfikacji, gdyż wylosowany w locie dla konkretnego użytkownika unikalny kod, jest mu w momencie wrogiego ataku z zewnątrz nieznanym!

```
<form method="POST">
    <!-- MUST HAVE każdej aktywnej paczki -->
    {% csrf_token %}
</form>
```

13. Sesje (Sessions) i Wiadomości Flash (Messages)

13.1. Przechowywanie danych tymczasowych (Sesja)

Żądania HTTP są "bezstanowe", ale niemal każda platforma potrzebuje zapamiętywania małych dawek danych konkretnego widza przez całą jego podróż u Ciebie. W PHP była to zmienna `$_SESSION`. W Django nazywa się to prosto: `request.session`.

Obiekt ten zachowuje się dosłownie jak "słownik" pythona. Zostaje zapisany w domyślnej bazie po zamknięciu widoku, a na maszynie klienta leci małe zaszyfrowane "Ciasteczko" z ID tej bazy.

Odczyt/Zapis w sesji:

```
def kup_produkt(request, id):
    # Dopisanie do koszyka w wirtualnej pamięci sesji (dla niezalogowanych!)
    koszyk = request.session.get('koszyk_z_zakupami', [])
    koszyk.append(id)

    # Zapis sesji w dictionary!
    request.session['koszyk_z_zakupami'] = koszyk
    return redirect('podsumowanie')
```

Sesje są idealne do tymczasowych akcji np.: zgody na RODO, tymczasowy koszyk dla gościa, wizardów wielostopniowych.

13.2. Wbudowany moduł wiadomości (Messages framework)

Bardzo przydatnym użyciem "Sesji" jest informowanie użytkownika o tym, że jego "rejestracja przebiegła pomyślnie" i ten komunikat musi zostać wyświetlony po przeładowaniu strony TYLKO JEDEN RAZ ("Wiadomości Flash"). Z pomocą przychodzi wbudowany moduł Messages!

Widok (generuje komunikat):

```
from django.contrib import messages

def zapisz_zmiany(request):
    # Zapisujemy informację do wyświetlenia wkrótce!
    messages.success(request, 'Poprawnie zaktualizowano profil.')
```

```
# Typy: messages.info, messages.warning, messages.error, messages.success
return redirect('profil')
```

Szablon w `base.html` (Konsumuje wiadomości i kasuje z pamięci):

```
{% if messages %}
<ul class="lista-alertow">
  {% for m in messages %}
    <!-- m.tags wypisze klasę zależącą od typu ('success', 'error') -->
    <li class="alert-{{ m.tags }}">{{ m }}</li>
  {% endfor %}
</ul>
{% endif %}
```

14. Sygnały w Django (Signals)

14.1. Co rozwiązują Sygnały

Framework potrafi wysyłać tzw. "Sygnały", jeśli zajdzie bardzo ważne systemowe zdarzenie, np.: "Ktoś się zalogował", "Właśnie usunięto post z bazy danych", "Wygenerowano model relacyjny przed jego fizycznym zapisem".

Często korzystamy z sygnału `post_save` (wysyłanego z automatu tuż po zrobieniu save na modelu). Możemy "Złapać sygnał" (Receiver) i wywołać w pełni darmową nieprzeszkadzającą funkcję! Służy to idealnie np. do generowania `Profilu` w momencie, kiedy sam formularz Admina wygeneruje nowego `Usera`.

14.2. Jak "Złapać sygnał" - tworzenie powiązań

Najlepiej zarejestrować taką metodę w pliku nazwanym np. `signals.py` umieszczonym przy Twoich aplikacjach, lecz żeby nie zagłębiać się w konfigurację `apps.py`, dla krótkich potrzeb można wpisywać to do podziemia w `models.py`.

```
from django.db.models.signals import post_save # Złap Zapis po wykonaniu
from django.dispatch import receiver
from django.contrib.auth.models import User
from .models import ProfilUzytkownika

# Deklarujemy "odbierającego" i doczepiamy do niego Sygnał 'post_save' pochodzący stricte od Modeli 'User'.
@receiver(post_save, sender=User)
def utworz_profil_uzytkownika(sender, instance, created, **kwargs):
    # argument 'created' = True, jeśli to pierwszy świeży wstawiony INSERT a nie Update
    if created:
        # Został powołany User! Tu robimy magiczne utworzenie mu na boku Profilu!
        ProfilUzytkownika.objects.create(user=instance)

# Kolejny sygnał (update'ujący profil, jeśli z jakiegoś powodu zostanie on edytowany i zapięty odgórnie)
@receiver(post_save, sender=User)
def zapisz_profil_uzytkownika(sender, instance, **kwargs):
    instance.profiluzytkownika.save()
```

Dzięki temu sprytnemu trikowi, jeśli ktokolwiek wywoła czysty bazodanowy `User.objects.create_user()` - w tle niewidzialnie zostanie stworzona poboczna relacyjna klasa, której zapominaliśmy wyprodukować w poprzednich stadiach programu!

15. Testowanie automatyczne (Testing)

15.1. Idea testów

Django dostarcza narzędzia z wbudowanym sercem Pythona: moduł `unittest`, co stanowi kompletną paczkę do zrobienia bezkosztowych sprawdzianów swoich klas.

Głównym centrum jest klasa `TestCase`, która potrafi samodzielnie dla potrzeb testów sklonować całkowicie pustą pamięciową bazę testową `sqlite3`, odpalić procedury Twoich modeli i usunąć te dane w pył po sekundzie działania bez ingerowania na Twoim `db.sqlite3` projektu.

15.2. Pierwszy prosty test

Twój system domyślnie tworzy plik `tests.py`. Skonfiguruj go!

`tests.py`:

```
from django.test import TestCase
from .models import Produkt

class SklepTestCase(TestCase):
    # Metoda odpala się ZAWSZE jako pierwsza w bloku by przygotować fałszywe zmienne (mocki/testowe obiekty).
    def setUp(self):
        Produkt.objects.create(nazwa="Lew", cena=400, aktywny=True)
        Produkt.objects.create(nazwa="Zebra", cena=100, aktywny=False)

    # Każda metoda z prefixem 'test_' zostanie z automatu sprawdzona przy skanie komendy
    def test_produkt_potrafi_sie_poprawnie_wypisac(self):
        # Wyciągamy na siłę fałszywy stworzony chwilę wcześniej obiekt
        lew = Produkt.objects.get(nazwa="Lew")

        # Oczekujemy że to, co dostaliśmy z metody modelu jest ZGODNE "Equal" z przewidywanym ciągiem:
        self.assertEqual(str(lew), "Lew (400 PLN)")
```

Odpal testy uderzając komendą dla test runnera Django (znajdzie każdy kod z tagiem 'test'):

```
python manage.py test
```

15.3. Testowanie Widoków - Client

`TestCase` dziedziczy potężne narzędzie o nazwie `Client` co przypomina imitację Twojej przeglądarki! Umożliwia ono pobieranie i uderzanie do tras URL po fałszywym statusie!

```
from django.test import TestCase
from django.urls import reverse

class WidokiTests(TestCase):
    def test_strona_glowna_istnieje_dla_goscia(self):
        # Narzędzie self.client generuje testowego "użytkownika", i odpala GET na trasie URL 'start'
        odpowiedz = self.client.get(reverse('start'))

        # Test zderzeniowy: Czy system wygenerował poprawny plik w formacie statusowym "200"?
        self.assertEqual(odpowiedz.status_code, 200)
```

```
def test_strona_nieznana_daje_zgodnie_kod_404(self):
    odpowiedz = self.client.get("/tajemniczastronawniebycie/")
    self.assertEqual(odpowiedz.status_code, 404)
```

Testowanie obwarowań to główna zasada zachowania bezpiecznego i pewnego jakościowo programu przy wielkich iteracjach poprawek deweloperskich.

16. Zaawansowane zapytania i Optymalizacja bazy danych

16.1. Problem N+1 zapytań

Gdy korzystasz z ORM bez ostrożności, bardzo łatwo wygenerować problem tzw. "N+1 zapytań". Jeśli mamy np. Model `Artykuł`, który ma ForeignKey `Kategoria`, to wywołanie `artykuł.kategoria.nazwa` zmusza bazę do wygenerowania DRUGIEGO, osobnego zapytania SELECT, pomimo że zrobiliśmy początkowy obiekt.

```
artykuly = Artykuł.objects.all() # +1 ZAPYTANIE (ściągnie 100 artykułów do pamięci)
for a in artykuly:
    # W pętli 100 razy strzelisz do bazy (N) by uzyskać kategorię tego rekordu! Morderstwo bazy.
    print(a.kategoria.nazwa)
```

16.2. select_related (dla relacji "Jeden")

Kiedy rozwiązujesz relację "naprzód" (czyli wędrujesz poprzez ForeignKey lub OneToOneField - relacja typu "Jeden"), ratuje nas klauzula `.select_related()`. Generuje ona automatycznie pod kluczem sprytny JOIN na poziomie bazy danych!

```
# TYLKO 1 ZAPYTANIE z wewnętrznym łącznikiem (INNER JOIN)!
# Baza wyśle do nas od razu paczkę sklejonych obiektów Artykułu ORAZ ich Kategorii
artykuly = Artykuł.objects.select_related('kategoria').all()

for a in artykuly:
    print(a.kategoria.nazwa) # Zmienna jest już w pamięci. Baza odpoczywa!
```

16.3. prefetch_related (dla relacji "Wiele")

Kiedy wyciągamy listę dzieci (np. "Wiele" tagów dla Artykułu - relacje ReverseForeignKey i ManyToManyField), JOINy SQL-owe spowodowałyby masakryczne zwielokrotnienie rekordów i pożary by RAM komputera. Tu wykorzystuje się `.prefetch_related()`. Metoda wykonuje dla wszystkich nadrzędnych obiektów **tylko jedno osobne drugie zapytanie** pobierając całą powiązaną paczkę do tabeli krzyżowej i sklejając kod lokalnie w Pythonie.

```
# To rozwiązanie to łącznie ZAWSZE równe 2 zapytania (Obojętnie czy artykułów będzie 5, czy 100 000).
artykuly = Artykuł.objects.prefetch_related('tagi').all()

for a in artykuly:
    print(a.tagi.all()) # Lista dzieci wyłapaną z pamięci komputera.
```

17. Formsety — wielokrotne formularze

17.1. Czym jest Formset?

Jeśli musimy na jednym ekranie zaktualizować na raz listę 5 Autorów i 10 Zdjęć, użycie 15 klasycznych `forms.Form` w szablonie i zwalidowanie ich po stronie serwera to absurd koderki. Formsety, to "zestawy formularzy" pozwalające wygenerować obiekty hurtowo z odpowiednimi indeksami!

```
from django.forms import formset_factory
from .forms import ZadanieForm

# Generuje siatkę składającą się równo z 3 formularzy typu ZadanieForm!
ZadanieFormSet = formset_factory(ZadanieForm, extra=3)
```

W widoku i w HTML operujemy już na 1 pliku Set, zamiast na pojedynczym formularzu:

```
<form method="post">
    {% csrf_token %}
    {{ formset.management_form }} <!-- ABSOLUTNY OBOWIĄZEK przy listach, dodaje inputy o ich liczności dla sprawdzeń POST! -->

    {% for form in formset %}
        {{ form.as_p }}
        <hr>
    {% endfor %}
    <input type="submit" value="Zapisz masowo">
</form>
```

17.2. ModelFormsets - Magia dla modeli hurtowych

Służą do bezpośredniej, hurtowej operacji CRUD na tabelach! Tworzy zestawy do tabel modeli:

```
from django.forms import modelformset_factory
from .models import Autor

# Utwórz modelowy formularz z siatką umożliwiającą w 1 request wstawienie autorów
AutorFormSet = modelformset_factory(Autor, fields=('imie', 'nazwisko'))

# Widok POST zamykający temat hurtowego update:
def hurtowy_widok(request):
    if request.method == 'POST':
        formset = AutorFormSet(request.POST)
        if formset.is_valid():
            formset.save() # Hurtowo dopisuje wszystkie formularze po 1 kliknięciu!
```

18. Paginacja — stronicowanie

Zwracanie 1000 artykułów na raz uśmierciłoby serwer i wyczerpało łącze na smartfonie. W `ListView` stronicowanie mamy wbudowane (tzw. `paginate_by = 10`), ale jako developer musisz umieć zaimplementować je sam w FBV!

`views.py`:

```

from django.core.paginator import Paginator
from django.shortcuts import render
from .models import Produkt

def strona_sklepu(request):
    lista_objektow = Produkt.objects.all()

    # Przekazujemy QuerySet i informujemy w jaką paczkę limitu rozkroić go.
    paginator = Paginator(lista_objektow, 25) # Rozdzielaj po 25 produktów na 1 podstronę

    # Odbierz z paska adresu parametr ?page= (Złota zasada internetu!)
    # Np. w wejściu: sklep.pl/produkty?page=2   Pobierze: "2"
    numer_strony = request.GET.get('page')

    # Skompresowana zmienna gotowa tylko w obrębie 1 fragmentu
    odcinek_produkow = paginator.get_page(numer_strony)

    return render(request, 'sklep.html', {'produkty': odcinek_produkow})

```

Złoty Snippet w HTML do dodania pod pętlą `for` (pasek paginacji):

```

<div class="pagination">
    <span class="step-links">
        {% if produkty.has_previous %}
            <a href="?page=1">&laquo; pierwsza</a>
            <a href="?page={{ produkty.previous_page_number }}">poprzednia</a>
        {% endif %}

        <span class="current">
            Strona {{ produkty.number }} z {{ produkty.paginator.num_pages }}.
        </span>

        {% if produkty.has_next %}
            <a href="?page={{ produkty.next_page_number }}">następna</a>
            <a href="?page={{ produkty.paginator.num_pages }}">ostatnia &raquo;</a>
        {% endif %}
    </span>
</div>

```

19. REST API i Django REST Framework (DRF) — w skrócie

Gdy Twoim frontendem przestają być szablony HTML (Template), a staje się oddzielna w Vue.js / React / Angular czy smartfonowa aplikacja na Androida/iOS, nie wchodzisz na tradycyjne strony z formularzami, ale pytasz o surowe dane, tzw. format wymiany JSON. Piszemy wtedy "API".

Podstawową procedurą do tego jest pakiet nie należący bazowo do systemu: `django-rest-framework`! Trzeba go doinstalować z polecenia `pip install django-rest-framework`.

Najważniejszą innowacją DRF jest `Serializer`. Bierze on złożone skomplikowane obiekty Pythonowe klasy ORM i jak maszyna do siekania mięsa zmienia w zwykłe tablice JSON-a!

```

from rest_framework import serializers
from .models import Artykuł

class ArtykułSerializer(serializers.ModelSerializer):
    class Meta:
        model = Artykuł
        # Odsiewamy do słownika tylko te zdefiniowane pola
        fields = ['id', 'tytuł', 'autor']

```

Wyświetlanie (przy pomocy DRF APIView):

```

from rest_framework.response import Response
from rest_framework.decorators import api_view

# To jest endpoint dla nowoczesnych bibliotek JS'owych uderzających paczką Axios:
@api_view(['GET'])
def pobierz_artykuly(request):
    posty = Artykuł.objects.all()
    # "many=True" bo wysyłamy całą listę, a nie 1 post
    serializer = ArtykułSerializer(posty, many=True)
    return Response(serializer.data) # Pakiet DRF automatycznie podmieni słownik na czysty tekst JSON-owy!

```

20. Architektura serwerowa i publikowanie systemu na świat

Aplikacja Django gotowa w folderze Twojego komputera lokalnego (`python manage.py runserver`) JEST OSTATECZNIE ZABRONIONA DO OTWARCIA DLA KLIENTÓW z powodów fatalnego zarządzania ruchem bezpieczeństwa, awaryjności i braku równoległości wątków. By stać się "na zewnątrz" należy:

1. **Przejsz na Gunicorn (albo uWSGI)** - Zamiast wbudowanego biednego serwera testowego, Django jest otaczane "Potężną otuliną wątkową", której zadaniem jest trzymać wirtualne środowisko pod stałym ciśnieniem. `pip install gunicorn` → i zamiast `runserver` na systemie używa się `gunicorn mojprojekt.wsgi:application`
2. **Podpiąć serwer Nginx / Apache** - Nawet z potężnym Gunicornem, odpalenie 1 małego obrazka obciąża zbytnio kod Pythonowy! Do tego z pomocą przychodzi tytan optymalizacji Nginx, który jest instalowany z przodu maszyny na domenę, "odbija" wszystkie uderzenia żądające zwykłych JPG i CSS od Twojej maszyny od razu posyłając paczki użytkownikowi prosto z dysku (`STATIC_ROOT`), a całą resztę (prośby o weryfikację, logowanie do bazy danych, szablony HTMLowe z wbudowanymi listami) przerzuca w trybie "proxy_pass" do serwera Gunicorn! To najwydajniejsza droga na świecie.
3. **Przejsz z SQLite na solidnego PostgreSQL'a** - Baza SQLite tworzy dosłownie "Jeden plik dyskowy - `db.sqlite3`" co daje jej świetną i natychmiastową użyteczność deweloperską. Niestety - kiedy w Internecie zaloguje się więcej osób i chociaż 3 naraz spróbują zapisać dany Post uderzając guzik POST w ułamku tej samej mikrosekundy, z powodu tzw. wskaźnika "Twardej Lockady", SQLite roztrzaska się - generując Database Is Locked. PostgreSQL to natywny potwór zarządzający setkami tysięcy transakcyjnych relacji!

Więc - przełączasz `settings.py` - ustawiasz zmienne ENV dla bazy PostgreSQL na chmurze i odpalasz projekt! Twój unikalny projekt Django leży online.

21. Praktyczne przykłady – tworzenie aplikacji od A do Z

Sama teoria to za mało, aby zrozumieć, gdzie dokładnie umieszczać poszczególne fragmenty kodu. Django ma mocno rygorystyczną strukturę. Poniżej znajdziesz trzy kompletne, gotowe do uruchomienia przykłady różnej wielkości, pokazujące krok po kroku budowę całej

aplikacji – od stworzenia projektu po szablony.

Zanim zaczniesz każdy z poniższych projektów, zakładamy, że masz już środowisko i utworzony projekt:

```
django-admin startproject mojprojekt .  
python manage.py startapp nazwa_aplikacji
```

Pamiętaj również, by zawsze na początku dodać `nazwa_aplikacji` do listy `INSTALLED_APPS` w pliku `mojprojekt/settings.py` !

Przykład 1: Prosta Lista Zadań (To-Do List)

Cel: Stworzenie najprostszej aplikacji z jednym modelem, która pozwala na dodawanie zadań i oznaczanie ich jako wykonane. Wykorzystuje widoki funkcyjne (FBV).

1. MODEL BAZY DANYCH (`TODO/MODELS.PY`)

Najpierw definiujemy, co chcemy przechowywać. Tu wystarczy treść zadania i status.

```
from django.db import models  
  
class Zadanie(models.Model):  
    tresc = models.CharField(max_length=200)  
    wykonane = models.BooleanField(default=False)  
    data_dodania = models.DateTimeField(auto_now_add=True)  
  
    def __str__(self):  
        return self.tresc
```

Po napisaniu tego kodu uruchamiamy w terminalu: `python manage.py makemigrations` oraz `python manage.py migrate` .

2. WIDOKI — LOGIKA BIZNESOWA (`TODO/VIEWS.PY`)

Tworzymy jeden widok, który będzie jednocześnie wyświetlał listę oraz przyjmował nowe zadania z formularza POST. W tym przykładzie pobieramy dane z formularza "ręcznie", by zrozumieć czysty HTML.

```
from django.shortcuts import render, redirect  
from .models import Zadanie  
  
def lista_zadan(request):  
    # Jeśli ktoś wcisnął przycisk "Dodaj" na stronie  
    if request.method == "POST":  
        nowa_tresc = request.POST.get('tresc_zadania')  
        if nowa_tresc:  
            Zadanie.objects.create(tresc=nowa_tresc)  
        return redirect('lista') # Odświeża stronę, chroni przed podwójnym dodaniem  
  
    # Jeśli ktoś tylko wchodzi na stronę (GET)  
    zadania = Zadanie.objects.all().order_by('-data_dodania')  
    return render(request, 'todo/lista.html', {'zadania': zadania})  
  
def oznacz_wykonane(request, zadanie_id):  
    zadanie = Zadanie.objects.get(id=zadanie_id)  
    zadanie.wykonane = True
```

```
zadanie.save()

return redirect('lista')
```

3. PODPIĘCIE URLI (`MOJPROJEKT/URLS.PY` I `TODO/URLS.PY`)

Podpinamy ścieżki, żeby przeglądarka mogła dotrzeć do naszych funkcji.

W `todo/urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.lista_zadan, name='lista'),
    path('zrobione/<int:zadanie_id>/', views.oznacz_wykonane, name='zrobione'),
]
```

W głównym `mojprojekt/urls.py` upewniamy się, że podpięliśmy plik `todo/urls.py`:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('todo.urls')), # Pusta ścieżka = strona główna to To-Do List
]
```

4. SZABLON WIDOKU (`TODO/TEMPLATES/TODOLISTA.HTML`)

W aplikacji `todo` tworzymy foldery `templates`, a w nim `todo`. Tam dodajemy HTML.

```
<!DOCTYPE html>
<html>
<head><title>Lista Zadań</title></head>
<body>
    <h1>Moje Zadania</h1>

    <!-- Formularz wysyłający metodą POST -->
    <form method="POST" action="">
        {% csrf_token %}
        <input type="text" name="tresc_zadania" placeholder="Co jest do zrobienia?" required>
        <button type="submit">Dodaj</button>
    </form>

    <ul>
        {% for zadanie in zadania %}
            <li>
                {% if zadanie.wykonane %}
                    <strike>{{ zadanie.tresc }}</strike>
                {% else %}
                    {{ zadanie.tresc }}
                    <a href="{% url 'zrobione' zadanie.id %}">[Zrobione]</a>
                {% endif %}
            </li>
        {% endfor %}
    </ul>
</body>
</html>
```

```
        {% endif %}

    </li>

{% empty %}

    <li>Brak zadań. Gratulacje!</li>

{% endfor %}

</ul>

</body>

</html>
```

Przykład 2: Blog z relacjami (Kategorie i Artykuły)

Cel: System Bloga pokazujący, jak stosować klucze obce (relacje) i jak elegancko korzystać z Django Formularzy oraz Widoków opartych na Klasach (CBV).

1. MODELE Z RELACJAMI (`BLOG/MODELS.PY`)

Relacja 1 do N: Kategoria ma wiele postów. Post należy do jednej kategorii.

```
from django.db import models

class Kategoria(models.Model):
    nazwa = models.CharField(max_length=50)

    def __str__(self):
        return self.nazwa

class Post(models.Model):
    tytuł = models.CharField(max_length=100)
    tresc = models.TextField()
    kategoria = models.ForeignKey(Kategoria, on_delete=models.CASCADE)
    data = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.tytuł
```

Nie zapomnij o komendach `makemigrations` i `migrate` !

2. KLASA FORMULARZA (`BLOG/FORMS.PY`)

Zamiast ręcznie odbierać dane w widoku, deklarujemy formularz, który zbuduje się automatycznie z Modelu.

```
from django import forms
from .models import Post

class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ['tytuł', 'tresc', 'kategoria']
```

3. WIDOKI OPARTE NA KLASACH - CBV (`BLOG/VIEWS.PY`)

Klasy oszczędzają masę pisania. Użyjemy `ListView` do wyświetlania i `CreateView` do formularza.

```

from django.views.generic import ListView, CreateView
from django.urls import reverse_lazy
from .models import Post
from .forms import PostForm

class ListaPostow(ListView):
    model = Post
    template_name = 'blog/lista.html'
    context_object_name = 'posty' # dostępna w HTML zmienna
    ordering = ['-data']           # od najnowszych

class DodajPost(CreateView):
    model = Post
    form_class = PostForm
    template_name = 'blog/dodaj.html'
    success_url = reverse_lazy('lista') # Zależność od adresów w urls.py

```

4. ADRESY URL (`BLOG/URLS.PY`)

Pamiętamy o `.as_view()` dla klas!

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.ListaPostow.as_view(), name='lista'),
    path('nowy/', views.DodajPost.as_view(), name='dodaj'),
]

```

5. SZABLONY (DZIEDZICZENIE)

Tworzymy plik bazowy wewnątrz `blog/templates/blog/base.html` :

```

<!DOCTYPE html>
<html>
<body>
    <nav>
        <a href="{% url 'lista' %}">Strona Główna</a> |
        <a href="{% url 'dodaj' %}">Napisz post</a>
    </nav>
    <hr>
    {% block content %}{% endblock %}
</body>
</html>

```

Szablon widoku dodawania posta (`blog/templates/blog/dodaj.html`):

```

{% extends 'blog/base.html' %}

{% block content %}
    <h2>Dodaj nowy artykuł</h2>

```

```

<form method="POST">
    {% csrf_token %}
    {{ form.as_p }} ←!— Automatycznie tworzy select z kategoriami i inputy z tekstem! →
    <button type="submit">Opublikuj</button>
</form>
{% endblock %}

```

Szablon widoku listy (`blog/templates/blog/lista.html`):

```

{% extends 'blog/base.html' %}

{% block content %}
    <h2>Wszystkie Wpisy</h2>
    {% for p in posty %}
        <article>
            <h3>{{ p.tytul }}</h3>
            <p><strong>Kategoria:</strong> {{ p.kategoria.nazwa }}</p>
            <p>{{ p.tresc }}</p>
        </article>
        <hr>
    {% endfor %}
{% endblock %}

```

Aby mieć jakiekolwiek kategorie na liście wyboru, najlepiej zarejestrować model *Kategoria* w pliku `blog/admin.py` i dodać je przez Panel Administratora `/admin/`.

Przykład 3: Miniforum — Rejestracja i Wątki Użytkowników

Cel: Pokazanie obsługi autoryzacji (logowanie/wylogowanie), ograniczenia dostępu oraz przypisywania dodawanych obiektów do zalogowanego konta.

1. MODEL WĄTKU Z AUTOREM (`FORUM/MODELS.PY`)

Będziemy korzystać ze zintegrowanego w systemie użytkownika Django.

```

from django.db import models
from django.contrib.auth.models import User

class Watek(models.Model):
    tytul = models.CharField(max_length=150)
    tresc = models.TextField()
    autor = models.ForeignKey(User, on_delete=models.CASCADE)
    utworzono = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.tytul

```

2. WIDOKI LOGOWANIA I REJESTRACJI (`FORUM/VIEWS.PY`)

Wykorzystujemy wbudowane mechanizmy.

```

from django.shortcuts import render
from django.views.generic import ListView, CreateView
from django.contrib.auth.mixins import LoginRequiredMixin
from django.contrib.auth.forms import UserCreationForm
from django.urls import reverse_lazy
from .models import Watek

class Rejestracja(CreateView):
    form_class = UserCreationForm
    template_name = 'forum/rejestracja.html'
    success_url = reverse_lazy('login') # po sukcesie → na stronę logowania

class PrzeglądajForum(ListView):
    model = Watek
    template_name = 'forum/index.html'
    context_object_name = 'watki'

# LoginRequiredMixin chroni przed wejściem przez gości
class UtworzWatek(LoginRequiredMixin, CreateView):
    model = Watek
    fields = ['tytuł', 'tresc'] # uwaga! NIE DAJEMY TU POLA AUTOR!
    template_name = 'forum/nowy_watek.html'
    success_url = reverse_lazy('index')

# Przechwytyjemy moment zapisu formularza aby WSTRZYKNĄĆ bieżącego usera
def form_valid(self, form):
    form.instance.author = self.request.user
    return super().form_valid(form)

```

3. KONFIGURACJA ADRESÓW (`MOJPROJEKT/URLS.PY` | `FORUM/URLS.PY`)

Do globalnego rutera w `mojprojekt/urls.py` musimy wpiąć wbudowane systemy autoryzacji:

```

from django.contrib import admin
from django.urls import path, include
from django.contrib.auth import views as auth_views

urlpatterns = [
    path('admin/', admin.site.urls),
    # Domyślne logowanie (szuka forum/logowanie.html)
    path('login/', auth_views.LoginView.as_view(template_name='forum/logowanie.html'), name='login'),
    # Wylogowanie
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),

    path('', include('forum.urls')),
]

```

A wewnątrz aplikacji `forum/urls.py`:

```

from django.urls import path
from . import views

```

```
urlpatterns = [
    path('', views.PrzegladaForum.as_view(), name='index'),
    path('rejestracja/', views.Rejestracja.as_view(), name='rejestracja'),
    path('nowy/', views.UtworzWatek.as_view(), name='nowy'),
]
```

Nie zapomnij też o dodaniu przekierowań w głównym pliku `mojprojekt/settings.py` !

```
LOGIN_REDIRECT_URL = 'index'
LOGOUT_REDIRECT_URL = 'login'
```

4. SZABLONY

Nawigacja uwzględniająca to czy jesteśmy zalogowani czy nie (`forum/templates/forum/index.html`):

```
<!DOCTYPE html>
<html>
<body>
    <header>
        <h1>Witamy na Forum</h1>
        {% if user.is_authenticated %}
            <p>Witaj, {{ user.username }}!</p>
            <a href="{% url 'nowy' %}">Zalóż nowy wątek</a> |
            <!-- Zabezpieczenie Django 5 wymusza na wylogowywaniu wysłanie forma POSTem -->
            <form action="{% url 'logout' %}" method="post" style="display:inline;">
                {% csrf_token %}
                <button type="submit">Wyloguj</button>
            </form>
        {% else %}
            <p>Jesteś niezalogowany.</p>
            <a href="{% url 'login' %}">Logowanie</a> |
            <a href="{% url 'rejestracja' %}">Rejestracja</a>
        {% endif %}
    </header>
    <hr>

    <h2>Ostatnie Dyskusje</h2>
    {% for w in watki %}
        <div>
            <h3>{{ w.tytul }}</h3>
            <small>Napisane przez: <b>{{ w.autor.username }}</b> ({{ w.utworzono|date:"Y-m-d" }})</small>
            <p>{{ w.tresc }}</p>
        </div>
    {% endfor %}
</body>
</html>
```

Formularze Rejestracji, Logowania i Nowego wątku mają identyczną, szablونową strukturę. Plik logowania

(`forum/templates/forum/logowanie.html`):

```
<h2>Zaloguj się</h2>

<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Wejdź</button>
</form>
```

Tym sposobem otrzymaliśmy pełnoprawne mini-forum, a Django oszczędziło nam pisanie setek linii bezpiecznego i skomplikowanego kodu autoryzacyjnego!

Przykład 4: Prosty Dziennik Szkolny (Od zera do finału)

Cel: Stworzenie w pełni funkcjonalnego "Szkolnego Dziennika Ocen" kompletnie od zera. Pokazujemy tu pełen proces: od komendy inicjującej projekt, przez konfigurację plików, po gotowy produkt. Aplikacja pozwoli dodawać uczniów oraz przypisywać im oceny.

KROK 1: Utworzenie i konfiguracja projektu

Najpierw w terminalu (wewnątrz środowiska wirtualnego) wydajemy polecenia:

```
django-admin startproject dziennik_projekt .
python manage.py startapp szkoła
```

Teraz rejestrujemy aplikację. Otwieramy plik `dziennik_projekt/settings.py` i w sekcji `INSTALLED_APPS` dopisujemy `'szkoła'`:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'szkoła', # ← Nasza nowa aplikacja
]
```

KROK 2: Projekt bazy danych (modele)

Otwieramy plik `szkoła/models.py`. Tworzymy model Ucznia i model Oceny (powiązanej z uczniem):

```
from django.db import models

class Uczeń(models.Model):
    imie = models.CharField(max_length=50)
    nazwisko = models.CharField(max_length=50)
    klasa = models.CharField(max_length=10)

    def __str__(self):
        return f"{self.imie} {self.nazwisko} ({self.klasa})"

class Ocena(models.Model):
    uczen = models.ForeignKey(Uczeń, on_delete=models.CASCADE, related_name='oceny')
    wartosc = models.IntegerField(help_text="Wpisz ocenę od 1 do 6")
```



```
przedmiot = models.CharField(max_length=50)
data = models.DateField(auto_now_add=True)

def __str__(self):
    return f"{self.wartosc} z {self.przedmiot}"
```

Zapisujemy zmiany w bazie danych, używając w terminalu:

```
python manage.py makemigrations
python manage.py migrate
```

KROK 3: REJESTRACJA W PANELU ADMINISTRATORA

Otwieramy plik `szkola/admin.py`, aby móc łatwo dodać uczniów przez wbudowany panel:

```
from django.contrib import admin
from .models import Ucen, Ocena

admin.site.register(Ucen)
admin.site.register(Ocena)
```

Możesz teraz utworzyć administratora komendą `python manage.py createsuperuser` i zalogować się pod adresem `localhost:8000/admin/`, aby ręcznie wprowadzić pierwszych uczniów.

KROK 4: TWORZENIE FORMULARZA (FORMS)

Aby dodawać oceny przez stronę internetową, potrzebujemy formularza. Tworzymy nowy plik `szkola/forms.py`:

```
from django import forms
from .models import Ocena

class OcenaForm(forms.ModelForm):
    class Meta:
        model = Ocena
        fields = ['wartosc', 'przedmiot']
```

KROK 5: WIDOKI I LOGIKA (VIEWS)

W pliku `szkola/views.py` tworzymy dwa widoki: jeden do wyświetlania wszystkich uczniów, drugi do widoku szczegółów ucznia (gdzie będziemy mogli też dodawać mu oceny).

```
from django.shortcuts import render, get_object_or_404, redirect
from .models import Ucen
from .forms import OcenaForm

# Widok 1: Lista wszystkich uczniów
def lista_uczniow(request):
    uczniowie = Ucen.objects.all().order_by('nazwisko')
    return render(request, 'szkola/lista.html', {'uczniowie': uczniowie})

# Widok 2: Szczegóły ucznia (lista jego ocen) oraz formularz dodawania nowej oceny
def szczegoly_ucznia(request, uczen_id):
```

```

uczen = get_object_or_404(Uczen, id=uczen_id)

if request.method == 'POST':
    form = OcenaForm(request.POST)
    if form.is_valid():
        # Przypisujemy ucznia do oceny przed ostatecznym zapisem do bazy
        nowa_ocena = form.save(commit=False)
        nowa_ocena.uczen = uczen
        nowa_ocena.save()
        return redirect('szczegoly', uczen_id=uczen.id)
    else:
        form = OcenaForm()

return render(request, 'szkola/szczegoly.html', {
    'uczen': uczen,
    'form': form,
})

```

KROK 6: PODŁĄCZENIE ADRESÓW (URLS)

Teraz łączymy widoki z adresami przeglądarki. W głównym katalogu projektu w pliku `dziennik_projekt/urls.py` dodajemy:

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('szkola.urls')), # Główna strona oddelegowana do aplikacji szkoła
]

```

Następnie tworzymy plik `szkola/urls.py`:

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.lista_uczniow, name='lista_uczniow'),
    path('uczen/<int:uczen_id>', views.szczegoly_ucznia, name='szczegoly'),
]

```

KROK 7: SZABLONY HTML (TEMPLATES)

W folderze `szkola` tworzymy podfoldery `templates/szkola/` (czyli ścieżka z plikami to `szkola/templates/szkola/`).

Tworzymy tam dwa pliki:

1. `lista.html` - Spis uczniów z linkami do ich szczegółów:

```

<!DOCTYPE html>

<html>

<head><title>Dziennik Szkolny</title></head>

<body>

    <h1>Lista Uczniów</h1>

    <ul>

```

```

        {% for u in uczniowie %}
            <li>
                <a href="{% url 'szczegoly' u.id %}">
                    {{ u.imie }} {{ u.nazwisko }} (Klasa {{ u.klasa }})
                </a>
            </li>
        {% empty %}
            <li>Brak uczniów w bazie. Dodaj ich przez panel /admin/</li>
        {% endfor %}
    </ul>
</body>
</html>

```

2. szczegoly.html - Strona ucznia, jego oceny i formularz:

```

<!DOCTYPE html>
<html>
<head><title>{{ uczen.imie }} {{ uczen.nazwisko }}</title></head>
<body>
    <a href="{% url 'lista_uczniow' %}">- Wróć do listy</a>
    <h2>{{ uczen.imie }} {{ uczen.nazwisko }} - Oceny</h2>

    <ul>
        {% for ocena in uczen.oceny.all %}
            <li><strong>{{ ocena.wartosc }}</strong> - {{ ocena.przedmiot }} <small>({{ ocena.data|date:"Y-m-d" }})</small></li>
        {% empty %}
            <li>Brak ocen. Uczeń jest nieklasyfikowany.</li>
        {% endfor %}
    </ul>

    <hr>
    <h3>Dodaj nową ocenę</h3>
    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Wystaw Ocenę</button>
    </form>
</body>
</html>

```

KROK 8: FINAŁ - URUCHOMIENIE!

Ostatni etap to uruchomienie serwera aplikacji w terminalu:

```
python manage.py runserver
```

Wejdź na <http://127.0.0.1:8000/>. Aplikacja jest gotowa – możesz dodawać uczniów przez Panel Administratora, a z poziomu witryny płynnie sprawdzać ich dzienniczek i na bieżąco wpisywać oceny!